Mapping the Unseen: What an LLM API Can and Cannot Reveal About Its Architecture

Abhyuday Patel[a,*], Ranjan Kumar Behera[a], Aditi Kumari[b], Pratik Chourasia[a]

[a] *Indian Institute of Information Technology, Ranchi, India*
[b] *Jamshedpur Women's University, India*

Abstract

What can an API tell you about the hidden system behind it? Most answers to that question are guesses. Ours is a measurement. We treat the problem as *functional architecture identification under partial observability*. In plain words: we ask which parts of a model's design can be worked out from the outside, through the API alone, and which parts cannot. Then we measure that boundary instead of reporting one best-guess architecture. The answer splits into four kinds of property. Some are *identifiable*: the API contract, the tokenizer's count family, the autoregressive-compatible generation contract, the width of the output bottleneck over a measured range, and the numeric grid of the values sent over the wire. Some are *cost-limited*: they can be separated in principle, and we *estimate* how much information each request buys on each channel (a per-channel discrimination density $\hat{\kappa}$) to say how many requests the job needs under the stated probe family. Some are *access-limited*: the API simply hides the channel we would need, as with top-5-only logprobs for width, buffered streaming for decode timing, and depth. And some are *provably non-identifiable*: we give two scoped equivalence witnesses, and we build one of them for real, a width-768 model and a width-1024 twin with zero observable difference. The instrument is an active probe library with an evidence-ranking engine. The measuring stick is ForensicsBench, a blind ground-truth benchmark that serves 26 known models (dense transformers, Mamba SSMs, mixture-of-experts, MLA, encoders, and an encoder-decoder, with dtype, batching, and router variants) behind one anonymous contract with frozen thresholds. On this ground truth the battery recovers the API contract (25/25), the tokenizer count-family (18/24 exact; 5 correct abstentions; 1 wrong), the autoregressive-compatible generation contract (25/25), the transformer-vs-SSM split (8/8 decided (11 abstained) of 19 eligible subjects, through a validated decode-state probe), the output-bottleneck width over a measured range (resolved through 2,048, with lower bounds at the acquisition limit and never over-resolved), and the wire grid (80% (25 subjects)). It is weak on MoE (88%) and blind to depth, and the theory says why each hole exists. The same battery then reconstructs GPT-4o-mini, DeepSeek-V3.2, and the JEV decision endpoint, with every component graded by the reliability its detector earned on ground truth. Baselines in the style of NightVision and LLMmap run on the same subjects. A live multi-seed planner study and a commit-reveal blind fixture suite complete the evaluation. The contribution is the measured identification boundary $[\omega]_{\mathcal{C},\mathcal{X}}$, not a claim of exact model reverse engineering.

Keywords: functional architecture identification, black-box forensics, system identification, identifiability, language-model serving, active probing

Companion artifact. Scanner, ForensicsBench harness and journals, frozen protocol, and reproduction scripts: https://github.com/AbhyudayPatel/mapping-the-unseen

*Corresponding author

Email addresses: abhyuday.2023ug4002@iiitranchi.ac.in (Abhyuday Patel), ranjan@iiitranchi.ac.in (Ranjan Kumar Behera), aditikumarikv4@gmail.com (Aditi Kumari), pratik.2023ug2051@iiitranchi.ac.in (Pratik Chourasia)

1. Introduction

Frontier models are rented, not downloaded. You send a request to an API and an answer comes back. The weights stay hidden. But the API is not a blank wall. Every answer is produced by a real pipeline: a tokenizer, an embedding table, a stack of attention or state-space blocks, an output projection, and a serving system wrapped around all of it. Each of these stages leaves small traces in what comes back: the token counts on the bill, the score vectors (logprobs), the timing, and the way errors are reported. This paper asks one quantitative question and answers it with a measurement:

> **What can an API reveal about a hidden computational system? Exactly how much of its functional architecture can be identified, and where does identification provably stop?**

The answer is not a yes or a no. It is a *four-way boundary*. Some properties are **identified** outright by sending controlled requests. Some are **cost-limited**: they can be separated in principle, but each request buys only a little evidence, so the real question is how many requests the job needs. We answer that with an estimate of the information one request carries, written $\hat{\kappa}$, under a stated family of probes. Some are **access-limited**: the API does not expose the channel at all. Top-5-only logprobs, buffered streams, and a deployment cap are all examples. And some are **non-identifiable**: two different architectures provably produce identical observations, and we show the pair explicitly. In formal terms, what we recover is the *equivalence class* $[\omega]_{\mathcal{C},\mathcal{X}}$, the set of all configurations that would look identical through the channels we have, and, for each property g, the *identified set* $\mathcal{I}_g(\omega; \mathcal{C}, \mathcal{X})$, the values of that property that remain possible. We never recover the literal configuration ω, and we do not pretend to.

Why a benchmark changes the claims. Earlier black-box architecture work reports demonstrations on subjects the authors chose themselves. We did something different. We took 26 known checkpoints, covering dense transformers, Mamba SSMs, mixture-of-experts models (including OLMoE-1B-7B), an MLA mechanics subject, masked encoders, and an encoder-decoder, plus variants that change only the number format, the batching, or the routing. We served all of them behind one API contract whose capabilities match a real production API. We froze the protocol and every detector threshold. Only then did we measure what the scanner gets right. The 26 subjects are built on 20 distinct base checkpoints; the rest are serving variants of the GPT-2 checkpoint (precision, quantisation, wire rounding, batching) plus a two-checkpoint router. So every accuracy interval in this paper is bootstrapped at the checkpoint level. Variants of one model are never counted as independent architectures.

Three results reshaped the paper. (i) The flagship "decoder-only 0.97" score from our own earlier version did not separate transformers from state-space models at all. Both are autoregressive decoders, so both pass that test. We built and validated a decode-state probe that does separate them (Sec. 6). (ii) Output-width recovery is exact up to the widest model we tested, as long as the probe uses more prompt rows than the model has width. Under top-5-only access it is empty. We prove why and then demonstrate it (Prop. 2). (iii) The domain-timing MoE detector is genuinely weak (accuracy 88%), so every production MoE claim is reported at that reliability.

Production case studies. We then point the battery at three live endpoints: GPT-4o-mini, DeepSeek-V3.2, and the JEV decision model. On each it reconstructs the input/output boundary, the tokenizer count-family, the autoregressive generation path, the numeric format on the wire, and the serving behaviours. Every claim imports its reliability from the benchmark. DeepSeek's MoE lean agrees with its published design. Its prefill time on fresh prompts grows more slowly than quadratically (exponent 0.57, 95% CI [0.41, 0.74], $n = 56$). That is compatible with its documented sparse attention, but it does not single it out, and our same-hardware control shows exactly why (Sec. 7.4).

The evidence hierarchy. The paper's strength runs from the top down. Tier 1 is the formal non-identifiability witnesses. These are mathematically clean. Tier 2 is detector validation against

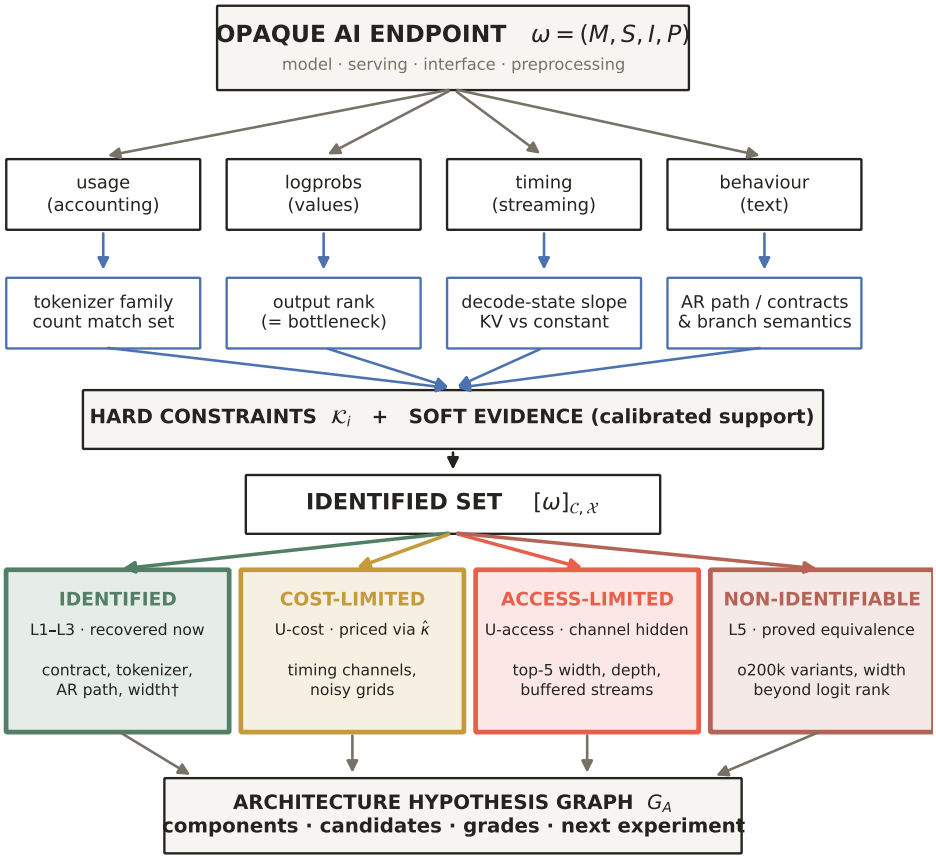


Figure 1. The whole paper on one page. An opaque endpoint $\omega = (M, S, I, P)$ is probed through four kinds of channel: usage accounting, logprobs, timing, and behaviour. Each observation becomes either a hard constraint or a piece of graded soft evidence. Together they define the identified set $[\omega]_{\mathcal{C},\mathcal{X}}$, which sorts every component into one of four classes: identified, cost-limited, access-limited, or witnessed non-identifiable. What remains is the architecture hypothesis graph G_A: an honest reconstruction, with a grade and a named next experiment attached to every node.

known ground truth on ForensicsBench. Here the instrument is measured, not merely plausible. Tier 3 is the controlled causal probes: transformer versus SSM, JEV branch isolation, deliberate serving changes, and same-hardware timing controls. Tier 4 is the interpretation of the three production systems. **The benchmark and the boundary are the contribution; the production systems are demonstrations.** They are graded demonstrations, because their detectors were measured first.

Contributions.

- **C1.** A clean statement of the problem: black-box architecture work is *functional architecture identification under API observability constraints*. This brings three tools: the equivalence class $[\omega]_{\mathcal{C},\mathcal{X}}$, the Forensic Layer Model (model $\neq$ serving $\neq$ interface $\neq$ preprocessing), and the four-way boundary. It also includes a channel-closure study showing that how much of an architecture is observable is itself a choice the API designer makes (Sec. 9).
- **C2.** ForensicsBench, a blind ground-truth benchmark: 26 served subjects, one anonymous contract, a frozen protocol, commit-reveal blind fixtures, and every journal released.
- **C3.** An evidence-ranking instrument that has been tested, not just described: evidence combination that respects dependence between probes, with measured calibration; a validated decode-state probe that separates transformers from SSMs; and width recovery with a measured coverage curve.

Table 1. The identification boundary, measured. Left: the functional components the battery identifies, with accuracy measured on ForensicsBench (known models served behind an anonymous API, with detector thresholds frozen in advance). Right: the components that stay open, each with the reason. The reason is always one of three things: a proved equivalence witness (grade L5), a channel whose per-request information κ we have estimated, or a named next experiment. Production rows are graded by how reliable the detector proved on the benchmark, never simply asserted.

Identified (with measured accuracy)	**Bench**	**Open, and why**
API contract and serving limits	25/25	depth: compute and queueing are tangled together; identity-layer witness (App. A)
Tokenizer count-family & template overhead	18/24 exact; 5 correct abstentions; 1 wrong	exact vocabulary: the reference set has gaps (DeepSeek: 465 unresolved entries)
AR-compatible generation contract vs encoder readout	25/25	—
Transformer vs constant-state (SSM) split	8/8 decided (11 abstained)	production span limit: decode growth is drowned in noise at $\leq$12k context (U-cost)
Output bottleneck width d (rich access)	15/15	top-5-only access: rank $\leq$ a handful of columns (witness, Prop. 2)
Wire numeric grid (fp32/bf16/fp16)	80% (25 subjects)	internal compute dtype: transport rounding hides it
Dense vs MoE feed-forward	88%	weakest detector; production claims discounted to match (Sec. 8)
Decision branch/readout structure (JEV)	$p = 2 \times 10^{-4}$	physical prefill sharing: only the operator can attest it

C4. Identifiability made practical: two scoped witnesses (one built for real, with zero observable difference), plus a query-cost bound with per-channel $\hat{\kappa}$ estimates, offered as a secondary and explicitly conditional tool (Sec. 4.3).

C5. Baselines (NightVision-style and LLMmap-style, run on the same subjects) and a live planner study. The planner study produced a negative result, and we report it: a simple static ordering is not beaten by our information-gain planner.

Scope. All probing used configured, authorised accounts with bounded budgets. We accessed no weights, no training data, and no private tenants. Dual-use considerations and release status are in Sec. 13.

2. The formal problem: what an API can and cannot see

2.1. Configuration, channels, and the identified set

Think of an endpoint as a bundle of four things, written $\omega = (M, S, I, P)$: the *model* itself, the *serving stack* that runs it, the *interface* that exposes it, and the *preprocessing* that prepares the input. When we send a request X_t, the API returns $Y_t = (y_t, \ell_t, u_t, t_t, h_t)$. Those five parts are the sampled text, the numeric values (logprobs or probabilities), the usage accounting, the timing, and the envelope (headers and error schemas). These are our only windows into the system.

Our target is not ω itself. It is the *equivalence class* $[\omega]_{\mathcal{C},\mathcal{X}}$: the set of every configuration that would be impossible to tell apart from ω when viewed through the channels $\mathcal{C}$ we have, using

only the requests $\mathcal{X}$ we are allowed to send. For each property g we also track the *identified set* $\mathcal{I}_g(\omega; \mathcal{C}, \mathcal{X})$: the values of g that remain possible (Def. 1 in Sec. 4). What we recover is *functional* architecture: how the input is tokenized, how text is generated, whether the model uses attention or a constant-size state, whether the feed-forward layers are dense or sparse, the geometry of the output, the numeric regime, and the serving structure. In other words, the organisation of the computation, as far as the equivalence class pins it down. Never weights, and never source code. Throughout the paper we call this *functional architecture identification under API observability constraints*. When we say "reconstruction", that is the shorthand.

Table 2. Plain-word glossary. The terms and grades used throughout the paper, in the order a reader meets them. Grades L1 to L5 and U are attached to every reconstructed component; the four boundary classes I, C, A, N are attached to every open question.

Term	**Meaning in plain words**
$\omega = (M, S, I, P)$	The hidden system: model, serving stack, interface, preprocessing.
channel	One kind of thing the API returns that we can measure: usage counts, logprobs, timing, behaviour, error text.
$[\omega]_{\mathcal{C},\mathcal{X}}$	All systems that would look identical to ω through our channels and allowed requests. This is what we can actually recover.
identified set $\mathcal{I}_g$	For one property g, the values that are still possible after everything we observed.
witness	A concrete pair of different systems that give identical observations. A witness proves a property cannot be identified.
κ, $\hat{\kappa}$	Information per request: how much evidence one API call gives for telling two candidates apart. High (around 2) means a few requests settle it. Low (around 0.02) means hundreds. Zero means never.
probe (P01 to P23)	A controlled request pattern designed to measure one thing, with a stated failure mode.
support score	The engine's ranking score for a candidate. It is a ranking, not a probability; we measure how far from a probability it is.
prefill / decode	Prefill is the work done on the prompt before the first output token. Decode is the per-token work after that.
KV cache / constant state	A transformer stores a growing key-value cache, so each new token costs a little more. An SSM keeps a fixed-size state, so it does not.
L1 observed	A value the API returned directly or that we measured directly (an error message, a token count).
L2 identified	A controlled experiment distinguished the stated alternatives.
L3 estimated	A numeric estimate with a model and an uncertainty or resolution statement.
L4 mechanism-consistent	The evidence favours one candidate, but alternatives remain.
U unresolved	The procedure, sample, channel, or budget was not enough to decide.
L5 non-identifiable	A scoped witness proves no admissible request can decide.
I / C / A / N	Boundary classes: Identified; Cost-limited (separable, priced by $\hat{\kappa}$); Access-limited (the API hides the channel); Non-identifiable (witness).

2.2. The Forensic Layer Model

The single most common mistake in informal probing is *layer confusion*: reading an interface fact as if it were a model fact. To guard against this we treat the four layers as separate causal zones, and every observation is tagged with the layer it is allowed to speak about:

$$\textit{model} \neq \textit{serving} \neq \textit{interface} \neq \textit{preprocessing}.$$

- A changed **output cap** is an interface change, not a model change. We demonstrate this with the cap fixture (Sec. 7.3).
- A change in **latency** says something about serving, and nothing about the model, until queueing, batching, and caching are controlled. The warm-versus-cold contrast in Fig. 9 is a factor of 5 to 10 on its own.

- **Token accounting** is a statement about the provider's ledger, not the tokenizer, until it is checked against a tokenizer we count locally. The ledger-inflation fixture shows why.
- **Logprob grids** are a statement about the wire format. The dtype used for the actual computation sits behind the rounding applied in transport. The dtype axis of ForensicsBench measures this gap.

A provider controls its own ledger [1]. A client's clock includes network noise. A deployment cap (16,393 tokens on GPT-4o-mini) is not the model's capability [2]. This layer discipline is what lets partial recovery stay honest. Each node of the hypothesis graph carries its provenance (provider-reported, payload-observed, client-measured, or independently verified) next to its evidence grade.

2.3. Access regimes are first-class

Every result is reported together with the access regime it used. *Standard* access means text, usage counts, top-k logprobs, and timing, which is what a typical production API exposes. *Restricted* access means top-5 logprobs only and no logit bias. *Rich* access adds a diagnostic selected-column channel that we built into our own serving wrapper, purely to measure what wider access would buy. The regimes are never mixed. Production claims use only standard access. ForensicsBench reports coverage for each regime separately (Sec. 6.2).

3. The forensic instrument

3.1. From requests to architectures: the hypothesis graph

The instrument works on one object, the *architecture hypothesis graph* G_A. It is built by a simple loop that anyone can audit:

request → observation → constraint → candidate architectures → next experiment.

We send a controlled request. It might be a pair of prompts that differ by one known string, so the token-count difference reveals how the tokenizer cuts text. It might be a timed decode at a known context length. It might be a request that deliberately asks for more output than the cap allows. Each request returns an observation. Each observation is then turned into a *constraint* on the configuration. When the observation model justifies it, the constraint is hard: a compatibility restriction $\mathcal{K}_i$ that rules candidates out entirely. An exact count match or a sharp rank cliff are examples. Otherwise the constraint is soft: a likelihood factor that ranks candidates without eliminating any, which is how a timing regime is treated. Hard constraints shrink the candidate set, $\mathcal{H}_t^{\text{hard}} = \mathcal{H}_0 \cap \bigcap_{i \le t} \mathcal{K}_i$. Soft evidence ranks whatever survives, $\hat{A}_t = \arg\max_{A \in \mathcal{H}_t^{\text{hard}}} S(A \mid D_t)$. In words: start from every architecture we know of, remove the ones the hard facts rule out, then score the rest.

The nodes of the graph are the components of the system: tokenizer, core family, attention or state regime, KV sharing, position scheme, feed-forward topology, output head, numeric regime, and serving topology. Each node holds the best-supported candidate, the full ranked list of candidates, the measured quantities, an evidence grade, provenance, and the *next experiment* that would separate the survivors. The edges are functional dependencies, each tagged with the experiment that produced it. The loop then spends the next request wherever the remaining ambiguity is largest (Sec. 3.4).

This graph is the unit of reconstruction. It never collapses an endpoint into a single label, and it never disguises a prior as a measurement. Where nothing can be measured, the node says so, lists the candidates, and names the experiment that would decide between them.

3.2. Probes and their failure modes

Table 3 organises the probe library by channel. Each probe declares an outcome model and returns three things: likelihood evidence, estimates with their assumptions spelled out, and a named falsifier (the observation that would prove it wrong). The full library is in the artifact.

Algorithm (the working progression, frozen per run).

1. **Discover the contract**: send a minimal request; then controlled invalid and over-limit variants; read the capabilities and error schema (P01/P05).
2. **Exploit the cheap channels**: usage accounting (P02 tokenisation deltas), wire numerics (P04), determinism (P10).
3. **Test the generation contract**: continuation length, whether latency grows with output length, streaming granularity. A rejected cap is itself evidence of an encoder (P19).
4. **Probe structure**: output geometry within the access regime (P03); decode-state slope (P23); needle retrieval and prefill scaling (P06/P07).
5. **Follow only unresolved branches**: each U node names its next discriminator; the planner prices it by expected information per request.
6. **Stop at the boundary**: an L5 witness, a $\hat{\kappa}$-priced cost, or a named experiment. Never a made-up label.

Figure 2. The operational progression. Every step names its failure mode. Nothing is asserted without a measurement that produced it.

Table 3. Probe families and the architectural constraint each one carries. The "remaining alternatives" travel with the claim into the graph, so a reader always sees what the probe could not rule out.

Probe	Constraint produced	Known failure mode (measured)
P01/P05	accepted-input boundary, output cap, error schema	a deployment cap is not the model's capability
P02 tokenizer	count-family match set, template overhead	finite strings cannot separate variants that share merge tables (Prop. 1)
P03 geometry	rank of the common-logit matrix ⇒ bottleneck estimate or bound	top-5-only access caps the rank at the number of columns seen (Prop. 2)
P04/P15	wire-grid / sampler signatures	rounding in transport is not the compute dtype
P06/P07	retrieval range, prefill/decode scaling	prefill curvature is invisible below about 16k context (Fig. 9)
P23	KV-growth vs constant-state slope (streaming per-token times or prefill-cancelling pairs)	drowned in noise on high-overhead production paths (reported U, not wrong)
P08–P11	drafting/cache/jitter/domain-regime signatures	domain timing is the weakest detector (88%)
P12–P14	capability/cutoff/format signatures	behaviour is not parameter count
P16–P22	routing/invariance/branch semantics	an unchanged API can hide a model swap (Sec. 7.3)

What matters for the claims in this paper is in the table, including each probe's failure mode as *measured* on ForensicsBench.

The discipline of negative inference. The instrument is built around what a channel does *not* say, because that is where informal probing goes wrong. Recall is not attention complexity: a wide sliding window and full attention both retrieve a needle. Latency is not kernel complexity: queueing time is not compute time. Behaviour is not a training recipe: post-training can rewrite behaviour without touching the architecture. A candidate that comes only from the prior is not recovered structure. And an unchanged API is not an unchanged model. Every evidence record carries its alternatives, and every figure draws prior-only values in grey.

3.3. Evidence combination that respects dependence

Each piece of evidence is a likelihood factor. For property j and candidate value v, the support is $q_j(v \mid D) \propto q_{j,0}(v) \prod_e \max(\Lambda_e(v), \epsilon)^{s_e}$. Read it this way: start from the prior $q_{j,0}(v)$, then for each evidence unit e multiply by that unit's likelihood $\Lambda_e(v)$, raised to a strength s_e that says how

much we trust the unit. The floor ϵ stops any single factor from driving a candidate to exactly zero. Version 2 of this paper multiplied every factor independently. That has a hidden flaw: a long series of similar probes gets counted again and again, and confidence inflates by repetition. The benchmark let us measure the damage. We now combine evidence *group-aware*: factors that come from the same experiment group (one probe series, one randomised block) are averaged into a single update. We report calibration for both schemes on ground truth (Table 7). Scores are still *support*, not probabilities, but the benchmark now tells us how far from a probability they are (expected calibration error 0.249) and where.

Two views accompany every posterior in the paper. The *reference-prior* view uses priors that encode the 2024–2026 model landscape. The *evidence-only* view starts from a uniform prior. Showing both separates what the data did from what the prior did. The gap matters most exactly where a strong prior meets a single decisive channel (Sec. 6.1).

3.4. The planner

The planner chooses the next probe by expected information gain per predicted request. In plain terms: for each probe it could run, it estimates how much the uncertainty would shrink (weighted by how diagnostic the probe is), divides by how many requests the probe would cost, and picks the best ratio. It is a heuristic scheduler, not an optimal Bayesian design. Sec. 7.2 evaluates it *live* against the simulator (not from replayed logs) with request budgets. That study found a real pathology, front-runner starvation, and the fix we adopted (the core-coverage floor).

4. Identifiability: the map of the boundary

We use identifiability theory the way engineers in system identification use it. It is not an excuse to give up. It is the instrument that tells us *which* holes in the reconstruction are fundamental, which are merely expensive, and which are one good experiment away.

Definition 1 (Observable equivalence and identified set)**.** *Take a configuration space Ω, a set of channels $\mathcal{C}$, and a set of admissible requests $\mathcal{X}$. Write $\omega_1 \sim_{\mathcal{C},\mathcal{X}} \omega_2$ when the two configurations produce the same distribution of responses for every request $x \in \mathcal{X}$. For a property g, the identified set is $\mathcal{I}_g(\omega; \mathcal{C}, \mathcal{X}) = \{g(\omega') : \omega' \sim_{\mathcal{C},\mathcal{X}} \omega\}$. A* non-identifiability witness *is a pair of equivalent configurations whose values of g differ, within the stated scope.*

In plain words: two systems are equivalent if no allowed request, read through the channels we have, can tell them apart. The identified set for a property is the list of values that property takes across all those look-alike systems. If the list has one entry, the property is identified. If we can exhibit two look-alike systems with different values, we have a witness, and the property is provably not identifiable through those channels.

4.1. Witness 1: token counts over ordinary text

Proposition 1 (Token-count equivalence, scoped)**.** *Under plain-text encoding with special tokens disabled, the cached* `o200k_base` *and* `o200k_harmony` *encoders produce identical token sequences, and therefore identical count observations, for every string in that domain.*

Proof. Both encoders use the same preprocessing regex and the same byte-pair merge ranks. Induction over the deterministic merge steps gives identical output. Disabling special-token interpretation removes the only difference between their dictionaries from the observation law. The artifact hashes the regex and every merge entry. □

The two tokenizers split ordinary text in exactly the same way, so they produce exactly the same counts. This is why GPT-4o-mini's tokenizer node resolves to the *o200k count family* rather than one specific encoder. The tie is structural. No amount of extra data breaks it. A third family built on the same tables would be just as invisible. Telling them apart would need observations on the special-token domain, and the API never emits those for ordinary text.

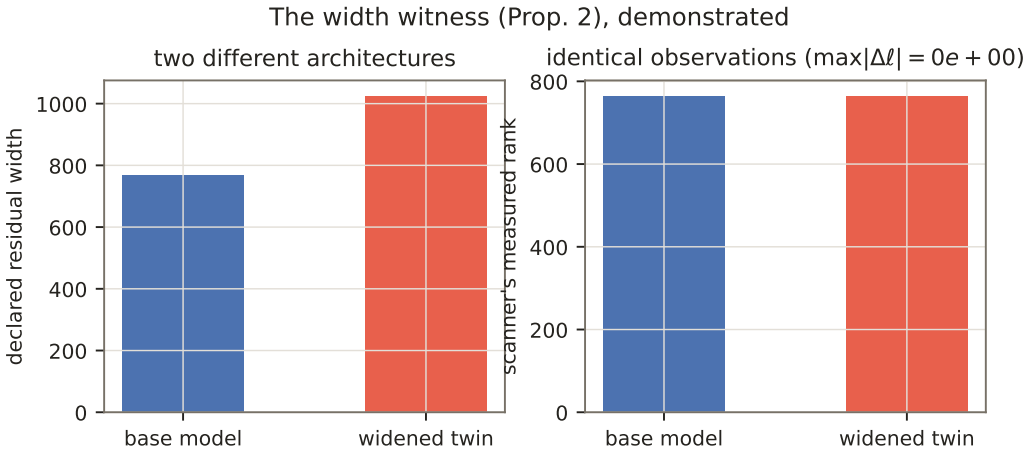


Figure 3. Prop. 2 demonstrated. A width-768 model and a constructively widened width-1024 twin produce bit-identical logit outputs ($\max|\Delta\ell| = 0$ over 48 prompts $\times$ top-20), and the scanner's geometry probe measures the same rank for both. Width beyond the logit rank is structurally non-identifiable.

4.2. Witness 2: width beyond the logit rank (constructive)

Proposition 2 (Output-rank width witness)**.** *Let M be a causal language model with final hidden width d, final readout $\phi(h)$, and unembedding matrix $E \in \mathbb{R}^{V\times d}$, so that the logits are $z = E\phi(h)$. For any $d' = d + k$, build a width-d' model with readout $\phi'(h) = [\phi(h); A\phi(h)]$ and unembedding $E' = [E \mid 0^{V\times k}]$. It produces identical logits on every input. Hence no channel that observes logits, full-vocabulary logprobs included and top-k all the more, can separate width d from width d'. Hidden width is identified only up to the* rank *of the observed logit matrix, and only under the extra assumption that the unembedding has full rank.*

Proof. $E'\phi'(h) = E\phi(h) + 0 \cdot A\phi(h) = E\phi(h)$. □

The construction is simple. Add k extra hidden dimensions, fill them with any function of the existing ones, and give the output matrix zero weight on all of them. Zero times anything is zero, so the logits never change. Nothing that looks only at logits can see the extra width.

We did not stop at the algebra. We built the twin: GPT-2 (width 768) and a width-1024 widened copy, both served behind the production contract. Over 48 prompts $\times$ top-20 logprobs the outputs agree exactly ($\max|\Delta\ell| = 0$), and the scanner's geometry probe measures the same rank for both (764 vs true 768/1024; Fig. 3). This is the formal content of the sentence "top-5 access cannot see width." With a top-k channel the set of columns we observe is limited to a handful of tokens, and the rank is capped by that set. Even with full logprobs the rank is still capped by the effective rank of the unembedding. Width recovery is therefore always a *rank statement plus an explicit full-rank assumption.* On production APIs the width question is closed by access, not by lack of cleverness.

4.3. The cost of separating what is *separable*

Some properties are not blocked by a witness. They are just expensive. To price them we use a single number per channel, κ, which we read as *information per request.* It says how much useful evidence one API query gives us for telling two competing hypotheses apart, measured in nats (the natural-log unit of information). If κ is high, say around 2, then the decode-timing question is a strong question for telling a transformer from an SSM: one or two requests settle it. If κ is low, say around 0.02, each request moves the needle only slightly, and we need many of them. If κ is exactly zero, no budget helps, because every allowed request returns the same answer under both hypotheses. The bound below turns κ into a minimum request count.

Table 4. Measured per-channel information per request, $\hat{\kappa}$ (nats per request, bootstrap 95% CI), for the hardest pair of hypotheses each channel must separate, together with the nominal lower-bound *scale* on request count at $\delta = 0.05$. The bound is conditional on the stated probe family: a point estimate of κ does not by itself prove the global KL domination the theorem assumes. The $\kappa = 0$ row is proved, not estimated.

Channel / hardest pair	$\hat{\kappa}$ (nats/req)	CI	nominal N scale (δ=.05)
Decode-state timing: full attention vs SSM (same hardware)	2.02	[1.85, 103.41]	1.3
Tokenizer counts: cl100k vs o200k (realistic prompt mix)	0.0177	—	150
Wire grid: fp32 vs bf16-rounded logprobs	deterministic grid	—	O(1)
o200k_base vs o200k_harmony, plain-text domain	0 (Prop. 1)	-	∞

Proposition 3 (Conditional query-cost lower bound)**.** *Let two candidates have conditional observation laws P_0^p, P_1^p for probe p, with cost $c(p) > 0$, and suppose every admissible conditional probe satisfies $D_{\mathrm{KL}}(P_0^p \| P_1^p) \le \kappa\, c(p)$. Then any adaptive test with type-I and type-II errors at most $\delta < 1/2$ needs $\mathbb{E}_0[\mathrm{cost}] \ge (1 - 2\delta) \log \frac{1-\delta}{\delta} \,/\, \kappa$.*

Proof. Apply the chain rule to the adaptive transcript (the policy itself adds no conditional divergence), bound each probe by κ times its cost, and pass the result through the binary decision by data processing. For random stopping, truncate and take limits. □

Here is what the bound says in everyday terms. The numerator is the amount of evidence a test needs to be wrong at most a fraction δ of the time. At $\delta = 0.05$ it works out to about 2.65 nats. Divide that by κ, the evidence per request, and you get the smallest number of requests any strategy can get away with. In version 2 of this manuscript the bound was stated without a value for κ: a shape without numbers. We now estimate κ for each channel from benchmark and serving data (Table 4). Under the stated probe families, the fitted discrimination models give a nominal lower-bound scale of about 150 requests at δ=0.05 to separate two near tokenizer families, against about 1.3 requests for the decode-state channel to separate attention from a constant state. Inside the domain of Prop. 1, $\kappa = 0$ exactly, so no budget helps. We treat $\hat{\kappa}$ as an empirical description of how much a channel is worth, not as a proof that the KL bound holds for every admissible probe. The guarantee is exactly as conditional as that assumption. In short, κ tells us how steep the boundary is.

The four-way boundary taxonomy. Every component in the paper carries one of four labels. **I** means identified: measured at grade L1 to L3. **C** means cost-limited: $\hat{\kappa}$ has been estimated, so resolving the component is a budget question with a computed price. **A** means access-limited: the channel we would need is not exposed on this interface, and the observability section (Sec. 9) measures what closing each channel costs. **N** means non-identifiable: a scoped witness exists, and no admissible request can help. This taxonomy replaces the vague phrase "future work" with a map you can act on.

5. ForensicsBench: a blind ground-truth benchmark

The core weakness of version 2 was simple: its production claims rested on detectors that had never been checked against a known answer. ForensicsBench fixes that. We serve *known* models behind the same anonymous contract the scanner faces in production, freeze the protocol, run the scanner, and only then look at the labels.

5.1. Subjects and axes

We serve 26 subjects (Table A.14). The dense decoders run from 128 to 2,048 in residual width: Pythia, OPT, GPT-2, DistilGPT-2, SmolLM, Qwen2.5-0.5B, and TinyLlama-1.1B. There

are two Mamba SSMs (130M and 370M), two MoE subjects (the trained OLMoE-1B-7B, and a random-init Mixtral mechanics subject), a random-init DeepSeek-V3-config MLA mechanics subject, four masked encoders from the BERT family, and one encoder-decoder (T5-small). Around this core we vary the *serving*, not the weights: the same checkpoint computed in fp32, bf16, and fp16; with its wire values rounded to bf16; with int8 dynamic quantisation; behind an 8-way batching condition; and a router fixture that alternates two checkpoints behind one model name. The random-init subjects exercise real code paths (MoE routing, MLA attention) even though their weights are meaningless. We label them mechanics subjects and disable the behavioural probes for them.

Effective sample size. The 26 subjects are *not* 26 independent architectures. They span 20 distinct base checkpoints. The serving axis adds five GPT-2 variant configurations (four completed; fp16 omitted) plus a two-checkpoint router. Those variants exist to measure serving confounds, not to inflate n. All headline accuracies therefore carry cluster-bootstrapped intervals at the base-checkpoint level (Table 6, 21 clusters), and we never quote the variant-inflated count as architectural coverage. 25 of 26 subjects completed the frozen protocol. The fp16-on-CPU variant ran out of time budget, because fp16 emulation on a CPU is several times slower than fp32 or bf16. We report it as omitted rather than quietly dropping it.

5.2. The anonymous contract

Every subject answers the same OpenAI-compatible `/v1/chat/completions` endpoint. It provides honest usage accounting, top-k logprobs with $k \leq 20$, `logit_bias`, SSE streaming with server-reported per-token times (in the style of Azure's `latency_checkpoint`), a stated context cap, and an explicit `measurement_columns` diagnostic that is used *only* in the rich-access regime. Ground truth is recorded from the loaded config classes, never typed by hand. The scanner sees no headers, no model strings, and no config hints.

5.3. Blindness and freezing

Three mechanisms keep the benchmark honest. (i) The battery protocol, meaning the probe set, the sample sizes, the spectral thresholds, and the decode-growth decision thresholds, is frozen in code and hashed (`protocol 31fc86fdc7fa99a8`) before any label is analysed. (ii) The controlled-change study is run commit-reveal. The harness draws each episode's label in secret, writes only a SHA-256 commitment to disk before scanning, and after the scan the reveal is verified against the commitment (Sec. 7.3). (iii) Detectors that failed during development are documented in the appendix together with their fixes. One was a curvature gate that fired without any effect-size check; another was a units error in the decode-growth slope. The benchmark reports the *repaired* instrument, and the failure log is part of the record.

5.4. What the benchmark measures

For each property we report three things on the support scores actually shown in this paper: top-1 accuracy, the Brier score, and the expected calibration error (ECE). Top-1 accuracy asks whether the highest-ranked candidate is right. The Brier score is the mean squared gap between the score and the truth, so lower is better and 0 is perfect. ECE asks whether a score of 0.8 is right about 80% of the time. All three are reported under two prior views (the reference prior and the evidence-only flat prior) and one sensitivity check (the legacy independent-factor combination). For each numeric target we report rank-recovery coverage by width and the relative error on resolved estimates, separately for rich and for production-realistic access. For each serving axis we report how the detectors respond to changes in dtype, batching, and routing. The baselines (Sec. 7.1) run on the same recordings.

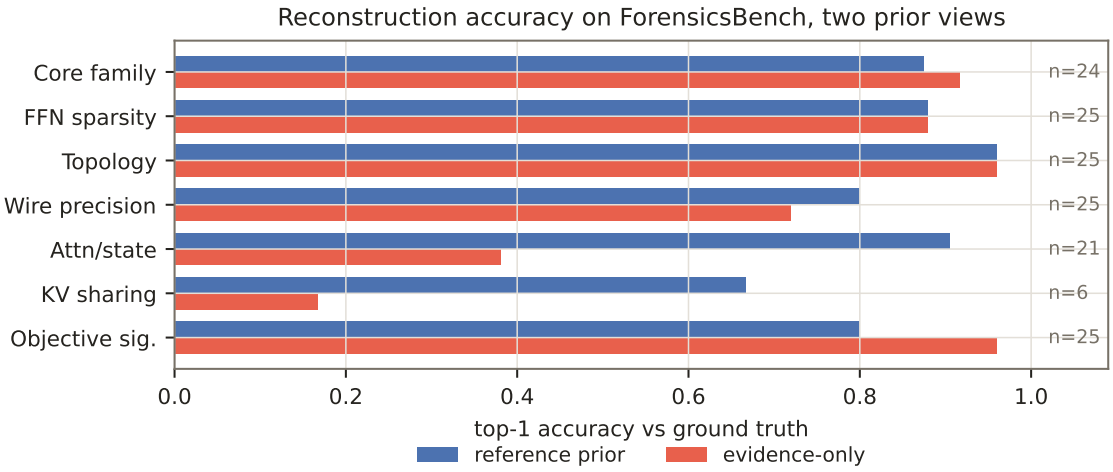


Figure 4. Per-property top-1 accuracy under the reference prior (blue) and the evidence-only flat prior (red). The two views disagree exactly where a strong prior meets a single decisive channel (core family, attention/state).

6. Detector validation: how far does reconstruction go?

Every number in this section comes from ForensicsBench subjects whose ground truth was recorded from the loaded configs. Detectors and thresholds were frozen before any label was analysed. Table 6 is the paper's central empirical object. Table 5 is its one-look summary: for each observable channel, the property it bears on, its identifiability class, its measured accuracy, and the main thing that can fool it.

Table 5. The observability matrix: what each API channel can establish about the hidden system, measured on ForensicsBench. **Class** is the boundary label of Sec. 4: **I** identified, **C** cost-limited ($\hat{\kappa}$ priced), **A** access-limited (channel absent or weak here), **N** non-identifiable (witness). Accuracy is top-1 against ground truth.

Channel	Property	Class	Accuracy	Coverage	Main confound
usage (accounting)	tokenizer count-family	I+N	18/24 exact, 5 abstain	registry-dependent	merge-sharing variants (Prop. 1)
generation	AR-compatible generation contract	I	25/25	all subjects	a 1-token output cap can mimic it
streaming timing	transformer vs SSM	C	8/8 decided	19 eligible; 11 abstain[§]	serving overhead at short spans
logprob geometry	bottleneck width	I[†]/A	15/15 core	rich: to 2,048; top-5: 0/25	rank $\leq$ observed columns (Prop. 2)
timing + jitter	dense vs MoE	C	88%	25 subjects	batching, cache, token mix
wire values	numeric grid	C	80%	25 subjects	transport vs compute rounding
behaviour	objective signature (gen/mask)	A	80%	25 subjects	post-training rewrites it
decision canaries	branch isolation	I[‡]	10/10 vs 0/10	JEV; perm. p=2×10^{-4}	physical readout open

[†]with the rich column channel; standard top-5 production access is class A. [‡]this endpoint.

[§]eligible = causal, non-router subjects: 8 decided + 11 abstained = 19.

n/a: 4 encoders, encoder-decoder, router; Table 6's n=21 adds recall on the last two.

Table 6. Reconstruction accuracy and calibration on ForensicsBench (top-1 of the support distribution against recorded ground truth). Both the reference-prior and the evidence-only (flat-prior) views are reported, because the difference between them is itself informative (Sec. 6.1). n varies: a property is scored only where ground truth is defined (for example, KV sharing has no meaning for encoders or SSMs). Top-1 intervals are cluster-bootstrapped at the base-checkpoint level, so serving variants of one checkpoint do not count as independent architectures (21 clusters).

Property	n	Top-1	95% CI[†]	Top-1 (evid.-only)	Brier	ECE	Note
Core family	24	88%	[70,100]	92%	0.300	0.169	P23 split inside; prior lag on SSM analysed in Sec. 6.1
FFN sparsity	25	88%	[71,100]	88%	0.436	0.335	weakest detector; production claims discounted
Serving topology	25	96%	[86,100]	96%	0.284	0.360	router fixture included
Wire precision	25	80%	[64,93]	72%	0.221	0.169	wire grid; int8 reads continuous by design
Attention/state regime	21	90%	[76,100]	38%	0.460	0.477	recall + decode-state, short spans
KV sharing	6	67%	[33,100]	17%	0.507	0.067	prior-only on production; bench measured where defined
Objective signature	25	80%	[60,95]	96%	0.230	0.164	causal vs masked vs prefix

[†]cluster bootstrap at the base-checkpoint level (serving variants of one checkpoint are one cluster).

6.1. The generation contract and the family split

The generation test establishes what we call the *autoregressive-compatible generation contract*. It looks at three things: how long a continuation the model produces, whether latency grows with output length, and how finely the stream is chunked. This is deliberately a claim about the contract, not the architecture. A serving wrapper that returns at most one token per request could produce the same observations with no autoregressive model behind it. So the architecture-level reading inherits that caveat, and the transformer-versus-SSM split below is what carries the architecture-level discrimination. Across ForensicsBench the contract row is correct on 25/25 subjects, including both Mamba SSMs. That is as it should be: SSMs *are* autoregressive at the contract level. Our earlier draft featured a "decoder-only 0.97" score that silently blurred this distinction. On the benchmark that score is exactly as uninformative as a sceptical reviewer would expect, because both SSM subjects score it too.

Telling a transformer from an SSM needs a different observable: the *decode state*. A transformer keeps a key-value cache that grows with every token, so each new token costs slightly more time than the last. An SSM keeps a state of fixed size, so the per-token time stays flat. The decode-state probe (P23) measures this slope from streaming per-token times at contexts of 64 to 2048 tokens, which is inside every deployment cap, taking the per-length minimum to suppress interference from other tenants. On ForensicsBench the split is 8/8 decided (11 abstained) of the 19 eligible subjects, meaning the causal, non-router ones. The remaining six of 25 are structurally ineligible: 4 encoders, the encoder-decoder, and the router (Fig. 9). On the same-hardware control it separates full attention (+18.6 µs per token per 1k of context, CI [16.3, 21.1]) from a 256-token sliding window (+1.8 [1.1, 2.6]) and from Mamba (+0.5 [−0.0, 1.2]). We ran 60 trials per arm; only 3 were needed for 80% power. Two honest qualifications. (i) Under the reference prior the categorical call lags behind the evidence. The prior odds against an SSM are 12 to 1, so a single decisive channel moves

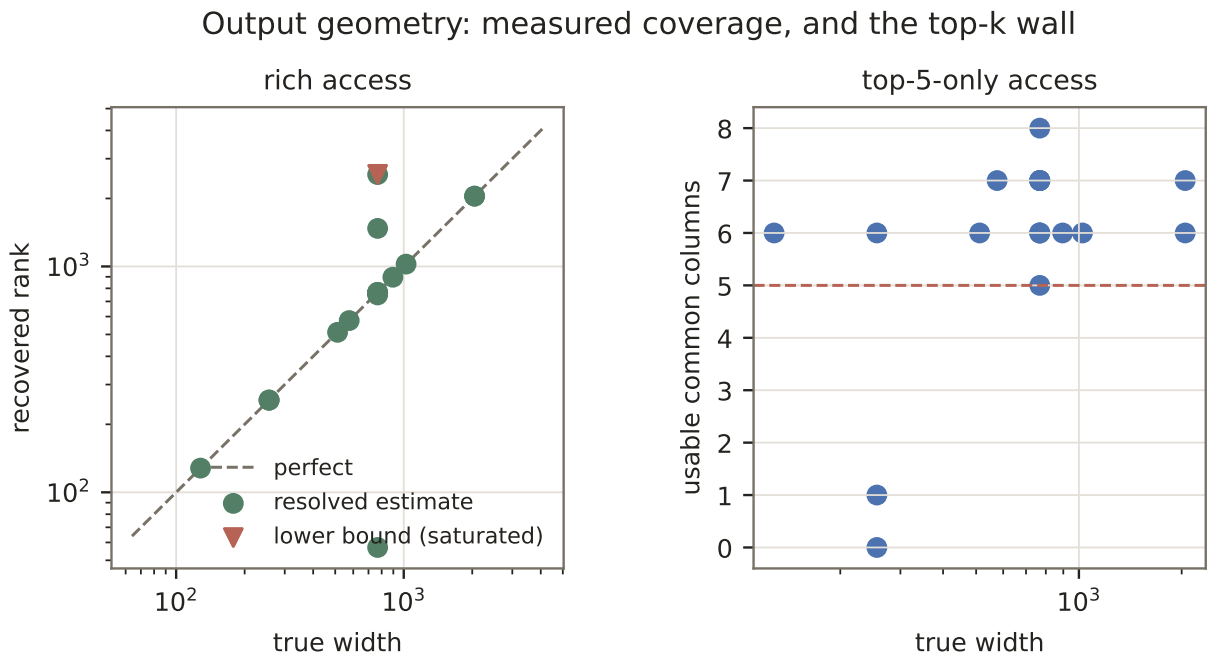


Figure 5. Left: recovered output-bottleneck rank against true width under rich access. The adaptive ladder resolves widths up to 2,048 and reports lower bounds when it saturates. The visible off-diagonal cases are serving-axis artifacts, discussed in the text: int8 quantisation deflates the measured rank (57 against a true 768), bf16 wire rounding inflates it (2552 against 768), and the two-checkpoint router roughly doubles it (1478). The geometry channel detects the routing fixture that the topology prior misses. Right: under top-5-only access the usable common column set is a handful. The rank is capped by the observed columns (Prop. 2), so width stays unresolved.

SSM to top-1 only in the evidence-only view. We show both. (ii) On production endpoints with high serving overhead the channel is drowned in noise at the spans we can reach, and there the probe abstains rather than guessing (Sec. 8).

6.2. Output geometry: coverage by width, and the top-k wall

With the rich selected-column channel, the geometry protocol now *adapts the number of prompt rows to the width it is looking for*. The ladder runs from 384 to 2560 rows, with a fixed relative threshold of 10^{-6} and a resolution gate fixed before measurement. The result is resolved estimates at widths 128, 256, 512, 576, 768, 896, 1024, 2048, which covers every core subject from 128 through 896, the 1,024-wide subject, and both 2,048-wide subjects, with relative errors under 2.6%, and lower bounds beyond that. At width 2,048 the ladder saturates at its 2,560-row cap and yields a lower bound rather than a point estimate. We report the bound. Under the production-realistic top-5-only regime, *no* subject resolves. The natural top-5 column union spans only 6 columns on average, so the matrix rank is capped by construction. That is Prop. 2 made empirical. This is the honest shape of the claim: **width recovery is a budgeted capability that depends on the access regime and comes with a measured coverage curve. It is not a demo.**

6.3. Numeric regime, tokenization, sparsity

The wire-grid detector scores 80% (25 subjects) across the dtype variants (fp32, bf16, and fp16 compute, plus the bf16-rounded wire). The int8-dynamic case reads as a continuous grid. That is correct at the wire layer, and it is a documented limit on inferring the compute dtype. Tokenizer count-family matching is 18/24 exact; 5 correct abstentions; 1 wrong exact within the reference registry. The domain-timing MoE detector is the weakest instrument, at 88% top-1: small dense models served on CPU produce MoE-like timing signatures often enough to matter. We therefore report MoE on production *at that measured reliability*, and the interleaved-control follow-up (Sec. 8)

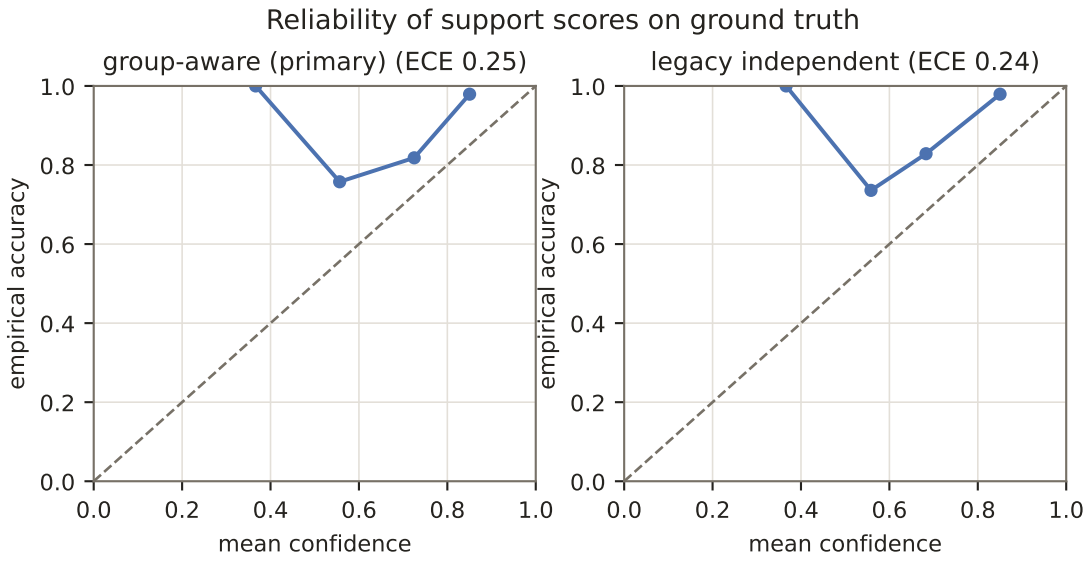


Figure 6. Reliability of support scores against ForensicsBench ground truth: group-aware ranker (left) versus the legacy independent-factor ranker (right).

replaces the domain-timing signal where it matters. The *objective signature* row of Table 6 measures the observable generation or masking contract (causal versus masked versus prefix behaviour), not the historical training recipe. Post-training can rewrite behaviour without touching the weights, so this row is named and graded as a signature, never as a recovered training objective.

The prior/evidence gap is a measurement. The attention/state property scores 90% under the reference prior but only 38% under the evidence-only flat prior. That is the largest prior-dependence anywhere in the benchmark. It is not merely a caveat. It is a result about forensic inference under weak channels: where the observable channel is thin (short-span recall plus a noise-limited decode slope), confidence in an architectural property is driven mostly by the structural prior. The honest report is the pair of numbers, not either one alone. We report both views for every property precisely so that prior-driven support is visible as such (Table 6).

6.4. Calibration: are the support scores honest?

Table 7 reports the Brier score and ECE of the scores shown in this paper, computed on ForensicsBench labels, for the group-aware ranker (primary) and the legacy independent-factor ranker (sensitivity). The seven properties differ in how many classes they have, how balanced those classes are, and how hard they are, so a single pooled number would mix unlike prediction problems. We therefore report the *macro-average* over properties with a property-bootstrap CI (ECE 0.249 [0.153, 0.351], Brier 0.348 [0.272, 0.429]) and the per-property values in Table 6. Grouping protects against mechanically compounding repeated evidence, which is the failure mode that motivated it. But it does *not* improve these aggregate calibration metrics here (legacy: ECE 0.244 [0.153, 0.346], Brier 0.344 [0.269, 0.423]). We report both schemes and claim no calibration improvement. Scores remain support ranks, never probabilities, and their rank order now carries a measured error rate per property (Table 6).

7. Experimental-design validation

Having validated the detectors, we now validate the *design* machinery around them: the planner that chooses experiments, the published baselines run on the same subjects, and the blind change-detection suite.

Table 7. Calibration of the support scores on ForensicsBench ground truth, macro-averaged over the seven heterogeneous properties of Table 6 (each weighted equally, since their class counts and difficulties differ; CIs by bootstrap over properties). Per-property values appear in Table 6.

Ranker	**Brier**	95% CI	**ECE**	95% CI
group-aware (primary)	0.348	[0.272,0.429]	0.249	[0.153,0.351]
legacy independent factors	0.344	[0.269,0.423]	0.244	[0.153,0.346]

Macro-averaged over the seven properties of Table 6 (equal weights);
CI by bootstrap over properties; per-property values in Table 6.

7.1. Baselines on the same subjects

Different methods solve different problems under different observation regimes. So we compare *on the same subjects, the same recordings, and the same access*, and we report each method against its own target rather than forcing a single leaderboard number. NightVision-lite estimates width from the rank of the top-5 column union and depth from inverted timing, holding hardware constant with a leave-one-out scheme. It resolves 0/25 widths, because its access regime cannot get past the top-5 wall either, and its depth estimates carry a median relative error of 30% on the subset where that estimator is defined. LLMmap-lite fingerprinting identifies 71% of subjects by nearest neighbour and classifies family at 35% (leave-one-out). Neither baseline is a strawman. Fingerprinting is the right tool for the question *which known model is this*, but it has no mechanism for an architecture outside its reference set, which is what the router and mechanics subjects are. NightVision-style estimation targets weight-level dimensions that we deliberately treat as access-limited here. Our pipeline reports structure with measured reliability and abstains where the channel is empty. Head-to-head numbers on the shared subset are in Table 8.

Table 8. Baselines on the same ForensicsBench subjects, recordings, and access regime. This is an apples-to-apples comparison on the shared subset, not a claim that one method dominates another: fingerprinting identifies known models, NV-lite estimates dimensions under top-5 access, and this paper reports partial identification with abstention.

Method	**Geometry**	**Identity/family**
NightVision-lite (top-5 width + timing depth)	0/2 widths	median depth err 30% (n=3*)
LLMmap-lite (24-prompt fingerprint)	identification 71%	family 35% (n=26)
This paper	geometry resolved 15/15; family 88%	graded support, abstention with named next step

Same subjects, same recordings, same access regime. *NV-lite's timing-inverted depth is defined only where its precondition holds.

7.2. The planner, live

We re-ran the planner comparison *live*, with no replay of saved logs: 8 seeds per subject per planner against the simulator, with per-seed serving jitter, request budgets, and a frozen recovery target. The honest headline is mixed, and we report it as measured. On requests-to-top-1 all three planners tie, because the baseline sweep already suffices on easy subjects. On requests-to-*decisive-discriminator evidence*, meaning how many requests it takes before the one probe that actually settles the family question has run, the simple cheapest-first static ordering is *not* beaten by the information-gain planner. Static lands the discriminator at 150 requests, against 185 for EIG and 233 for uniform ordering on the family-split subjects. The reason is mundane: the cheap probes happen to include the discriminators. The pathologies are the real findings. (i) A pure EIG surrogate starves the very probe that could unseat a confident front-runner, because the surrogate assigns the upset a low probability and so expects little gain from it. The engine now carries

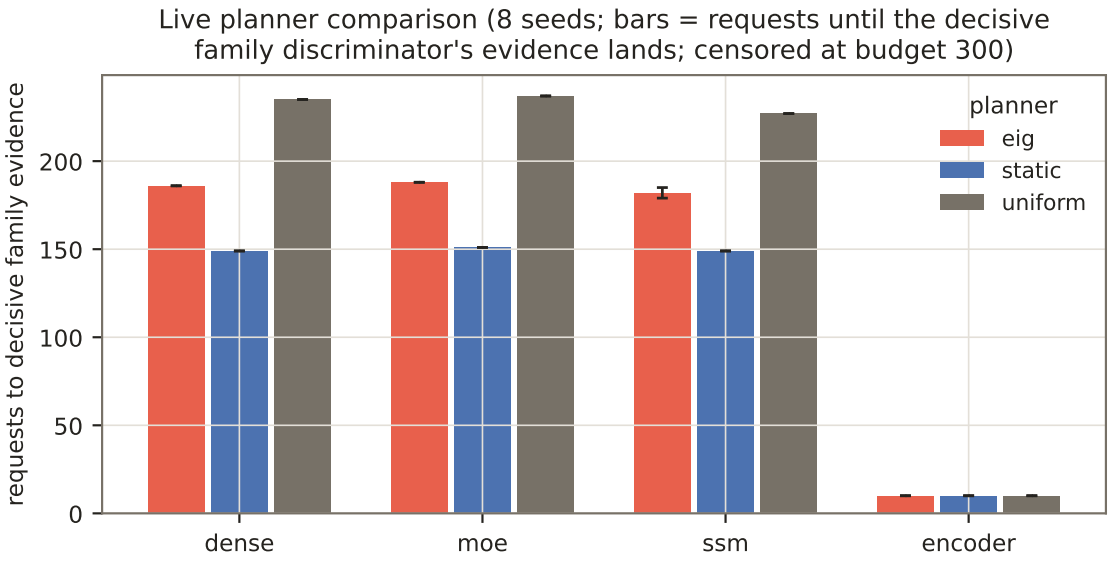


Figure 7. Live planner comparison (8 seeds, per-seed serving jitter). Requests until the decisive family discriminator's evidence lands.

a *core-coverage floor*: before the diminishing-returns stop fires, at least one high-diagnosticity challenger must have run for every core property. Without that floor, EIG never runs the decisive probe on the SSM subject at all. (ii) Under the strong reference prior, the SSM target stays uncrossed within budget for every planner. Planning cannot beat a prior; only evidence can (Sec. 6.1). Table 9 has the full matrix, and Fig. 7 shows the per-subject picture.

Table 9. Live planner comparison (no replay): requests to the top-1 family target and to arrival of decisive family evidence (P23/P19-prefill), mean [95% CI] over 8 seeds, † = censored at the 300-request budget.

Subject	Planner	req → top-1	req → decisive	censored
dense	eig	8 [8,8]	186 [186,186]	0%
dense	static	8 [8,8]	149 [149,149]	0%
dense	uniform	8 [8,8]	235 [235,235]	0%
encoder	eig	10 [10,10]	10 [10,10]	0%
encoder	static	10 [10,10]	10 [10,10]	0%
encoder	uniform	10 [10,10]	10 [10,10]	0%
moe	eig	8 [8,8]	188 [188,188]	0%
moe	static	8 [8,8]	151 [151,151]	0%
moe	uniform	8 [8,8]	237 [237,237]	0%
ssm	eig	300 [300,300]	182 [179,185]	0%
ssm	static	300 [300,300]	149 [149,149]	0%
ssm	uniform	300 [300,300]	227 [227,227]	0%

7.3. Blind controlled changes

The fixture suite is now a blind test. The harness draws each episode's label and commits it as a SHA-256 hash before the scan. The detectors are frozen. After the scan, the reveal is checked against the commitment. Over 48 episodes the sensitivities are: cap change 100%, ledger inflation 100%, wire rounding 100%, checkpoint swap 100%, and router alternation 100%, with 100% specificity on unchanged controls (Wilson 95% CIs in Table 10). These are *large, designed* changes. The suite measures the detection floor, not the rate of wrong claims on a real population.

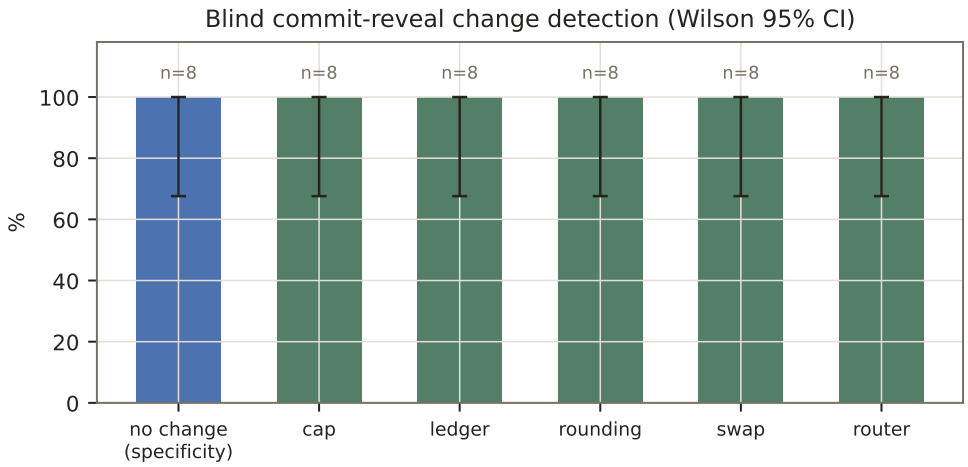


Figure 8. Blind commit-reveal change detection (Wilson 95% CIs).

Table 10. Blind commit-reveal fixture results (detectors frozen; labels committed by hash before each scan).

Episode	**Rate**	**Wilson 95% CI**	n
no change (specificity)	100%	[68,100]%	8
output-cap change	100%	[68,100]%	8
ledger inflation	100%	[68,100]%	8
wire rounding	100%	[68,100]%	8
checkpoint swap	100%	[68,100]%	8
router alternation	100%	[68,100]%	8

7.4. Same-hardware attention control

Figure 9 is the control that the production DSA interpretation needed. We served three stacks on identical hardware: full attention (GPT-2), a sliding window of 256 tokens (Mistral-config), and a constant state (Mamba-config), and measured all three with the production protocol. Decode growth separates the mechanisms decisively. Fresh-prefix prefill exponents at $\leq$1k context are all between 0.8 and 0.9, which is sub-quadratic for *every* mechanism at that span. That is why any claim based on prefill curvature needs long context. The warm-versus-cold prefix contrast is a factor of 5 to 10, which shows how large the cache effect is that contaminated naive fresh-versus-repeat comparisons. Every production timing claim in Sec. 8 is graded against this control.

8. Production reconstructions, graded by measured reliability

We now return to the three production endpoints. They are *demonstrations* of the measured instrument, not the contribution itself. Every component below carries three things: (i) its evidence grade, (ii) the probe that produced it, and (iii) that probe's measured reliability on ForensicsBench from Sec. 6. One caveat applies to everything in this section. The reliabilities were measured on ForensicsBench under its own observability and serving regime. Carrying those error rates over to a frontier production deployment is an assumption about external validity, because the benchmark and production are different populations: CPU serving against accelerated serving, and known checkpoints against frontier stacks. In symbols, $P_{\text{production}} \neq P_{\text{benchmark}}$.

8.1. GPT-4o-mini: the worked reconstruction

The eight-step progression (Fig. 11) shows the instrument at work on an opaque endpoint. Each step says what it can establish and what it cannot.

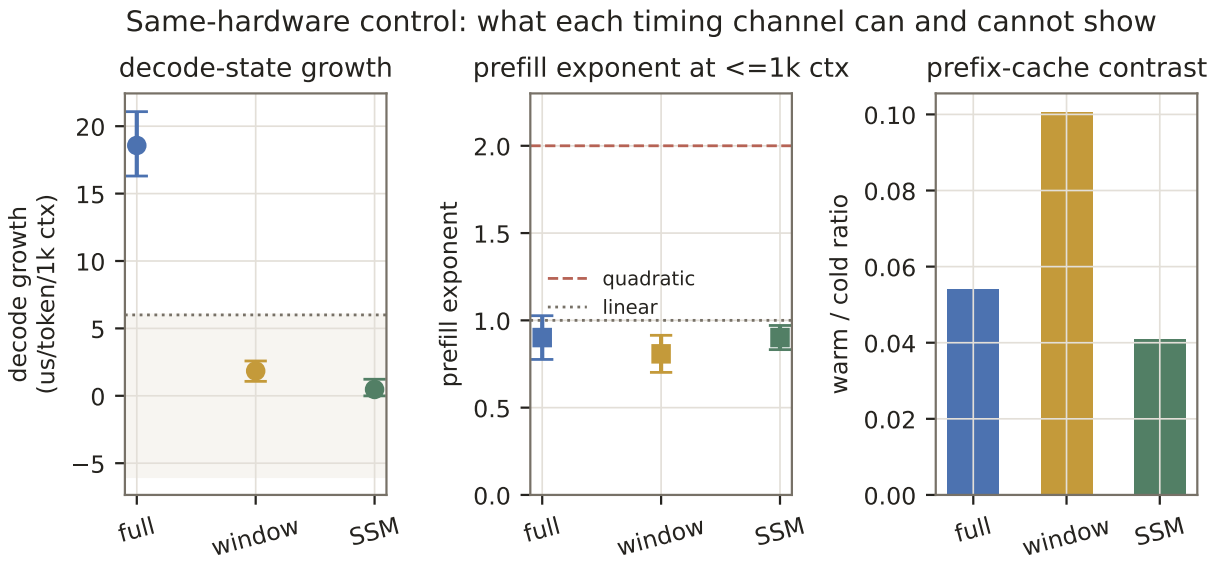


Figure 9. Same-hardware control (GPT-2 full attention; Mistral-config 256-token sliding window; Mamba-config constant state; 60 trials per arm, ≥3 needed for 80% power). **Left:** decode-state growth separates the three mechanisms cleanly. **Middle:** fresh-prefill exponents at ≤1k context are sub-quadratic for *all* mechanisms, so prefill curvature tells you nothing at short span. **Right:** the prefix-cache contrast (warm/cold 0.04–0.10) is the effect that contaminated naive fresh-versus-repeat comparisons.

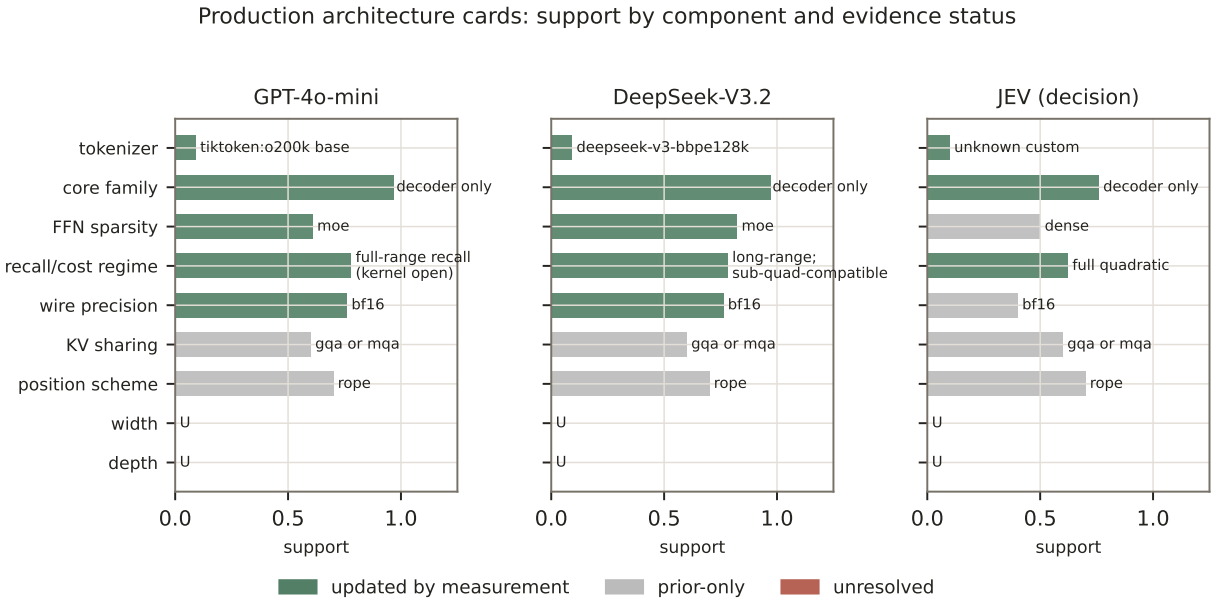


Figure 10. Production architecture cards: per-component support with evidence status (green = updated by measurement, grey = prior-only, red/U = unresolved). Generated from the recorded scan posteriors; the attention row carries the powered follow-up's reading, not the recall-only prior.

1. **Interface boundary.** An over-limit request reads the deployment contract straight from the error text: 16,384 maximum completion, 16,393-token accepted input (L1). The model's documented capability is larger [2]. This is a fact about the interface layer, not the model (Sec. 2.2).
2. **Tokenization.** Count differences on a fixed scaffold match `o200k_base` and `o200k_harmony` on all 18 strings. That is a count-family identification with a proved tie (Prop. 1), L2.
3. **Generation path.** A 24-token continuation whose latency rises with output length ($23 \to 202$ ms) establishes the autoregressive-compatible generation contract, L2. This is

contract-level, not transformer-specific (Sec. 6.1).

4. **Attention/state regime.** 18 of 18 needle retrievals succeed through 8,225 tokens, which establishes full-range recall. The attention kernel itself stays unseparated at these spans (L4; Fig. 9).
5. **KV sharing / positions.** GQA/MQA and RoPE appear only as priors. They are shown in grey and never counted as recovered. The discriminators that would decide them are recorded by name.
6. **Numeric regime.** The logprob lattice is consistent with bf16 (support 0.76), L4 at the wire layer.
7. **Feed-forward.** Dense or MoE is ambiguous (0.61 against 0.39). It is reported at the detector's measured 88% reliability, with the interleaved control returning a null.
8. **Width/depth.** Width is unresolved by access (the top-5 wall, Prop. 2). Depth is unresolved by confound (compute and queueing are tangled). Both are U with named next experiments.

Table 11. GPT-4o-mini: reconstructed components. "Bench" is the detector's measured accuracy on ForensicsBench; grey rows are prior-only (no updating evidence).

Component	Reconstruction	Grade	Bench	Basis
Input/output boundary	16,393-token deployment cap; 16,384 max completion	L1	-	error text; docs note larger model capability
Tokenizer	o200k count family (base=harmony tie)	L2	18/24 exact	18/18 fixed-scaffold count matches (Prop. 1 tie)
Generation path	AR-compatible contract	L2	25/25	24-token continuation; latency scales with output
Transformer vs SSM	transformer-consistent, short span	U	8/8 decided	engine decode TBT 11 ms flat, CI $[-74, +86]$ µs/tok/1k (noise-dominated)
Attention regime	full-range recall to 8,225 tokens	L4	-	18/18 needle conditions; prefill exponent 0.17 [-0.01, 0.36] at $\leq$15k
Wire numerics	bf16-consistent lattice	L4	80%	P04 grid match
Feed-forward	dense/MoE ambiguous (MoE 0.61)	U	88%	domain timing weakest; interleaved control null
KV / position	GQA/MQA, RoPE (priors only)	U	-	no updating evidence; named next experiments
Width / depth	unresolved	U	-	top-5 access wall (Prop. 2); compute×queueing confound

8.2. DeepSeek-V3.2

The same battery reconstructs a BBPE count-family tokenizer (19 of 19 matches), an autoregressive decoder path, and a stated 131,072-token context. Two claims carry the paper's production weight.

MoE-leaning feed-forward. The domain and jitter signatures give MoE 0.82 against dense 0.18, and the published design is indeed MoE. But our own benchmark says this class of detector is only 88% accurate. So we report the lean *and* the discount together. The interleaved, length-matched, one-token control (10 randomised blocks with domain-label permutation) does not reject the null of no domain effect on either endpoint, which weakens the timing discriminator. The jitter signature remains. Grade: L4, mechanism-consistent with an independent published anchor. Not L2.

Fresh-prefix prefill scaling. The v2 comparison mixed a fitted repeated-prefix curve with eight unfitted fresh-nonce points. Most of what it showed was a prefix-cache artifact. The powered follow-up (Fig. 13) fits both curves with the same log-log model on randomised trials. A log-log fit means we plot the logarithm of time against the logarithm of context length; the slope of that

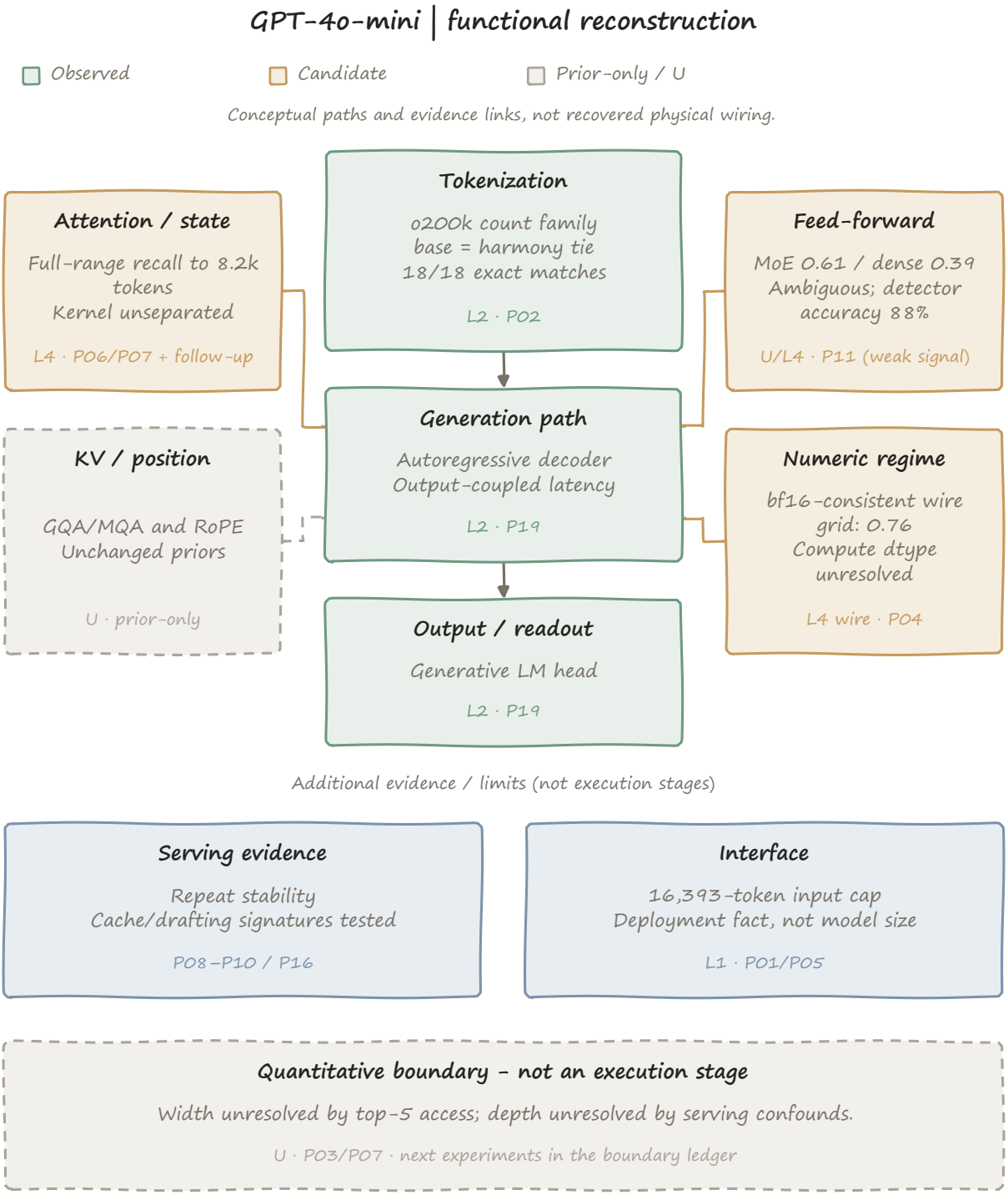


Figure 11. GPT-4o-mini functional reconstruction. Cards distinguish observed properties, mechanism candidates, unchanged priors, and unresolved components. Arrows show conceptual functional order; side links associate evidence with a component, not measured physical wiring. The quantitative boundary is separate from the path.

line is the scaling exponent, where 1 means linear and 2 means quadratic. The fresh exponent is 0.57 (95% CI [0.41, 0.74], $n = 56$); the repeated-prefix exponent is 0.22 ([0.04, 0.40], $n = 21$). The fresh curve is sub-quadratic, which is compatible with the documented sparse attention [3]. But our same-hardware control (Fig. 9) shows that small dense models at short spans also look sub-quadratic. So the exponent does not *select* DSA. It narrows the attention node to candidates that are compatible with sub-quadratic scaling. Grade: L4, not L2.

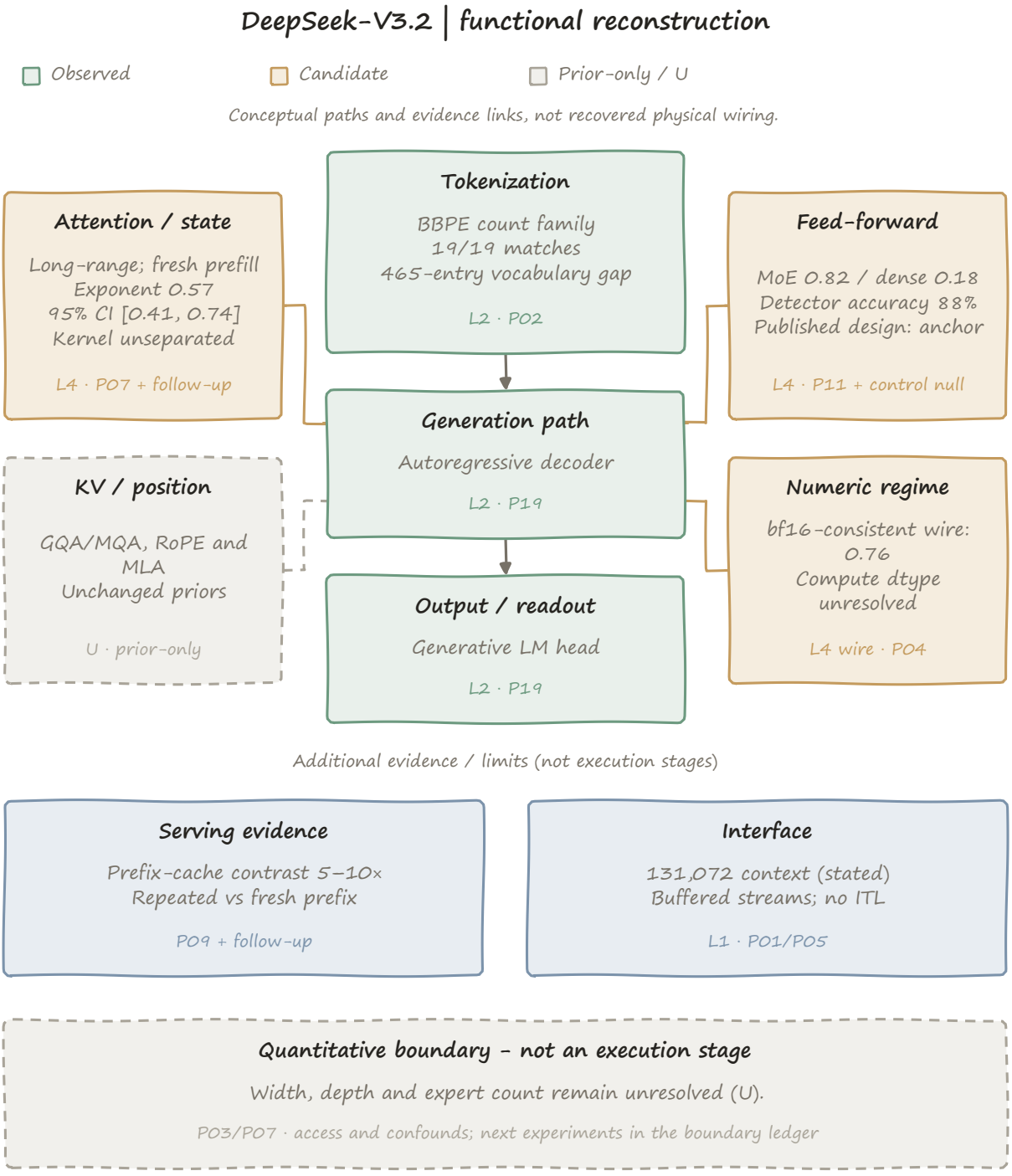


Figure 12. DeepSeek-V3.2 functional reconstruction: MoE-leaning feed-forward and long-context evidence with unresolved alternatives. Side links are evidence associations, not physical wiring; the published design is an evaluation anchor, not a measurement. Width, depth, and expert count remain unresolved.

8.3. JEV: a cross-contract generality test, powered

JEV plays one specific role in this paper. It is the *cross-contract generality test*: proof that the instrument is not a trick that only works on text-generation APIs, but also works on a decision endpoint with a different interface contract. Its reconstruction is unchanged in kind and stronger in evidence. The shared-state accounting follows an exact linear ledger, $u(q, s) = 259 + \text{state_cost}(s) + 12.0q$ with $R^2 = 1.00$: a fixed 259 tokens, plus the cost of the state, plus 12 tokens per question. The branches are isolated, the semantic choices are normalized, and the readout is a fixed 38 tokens. In v2 the branch-isolation claim rested on three trials. The powered

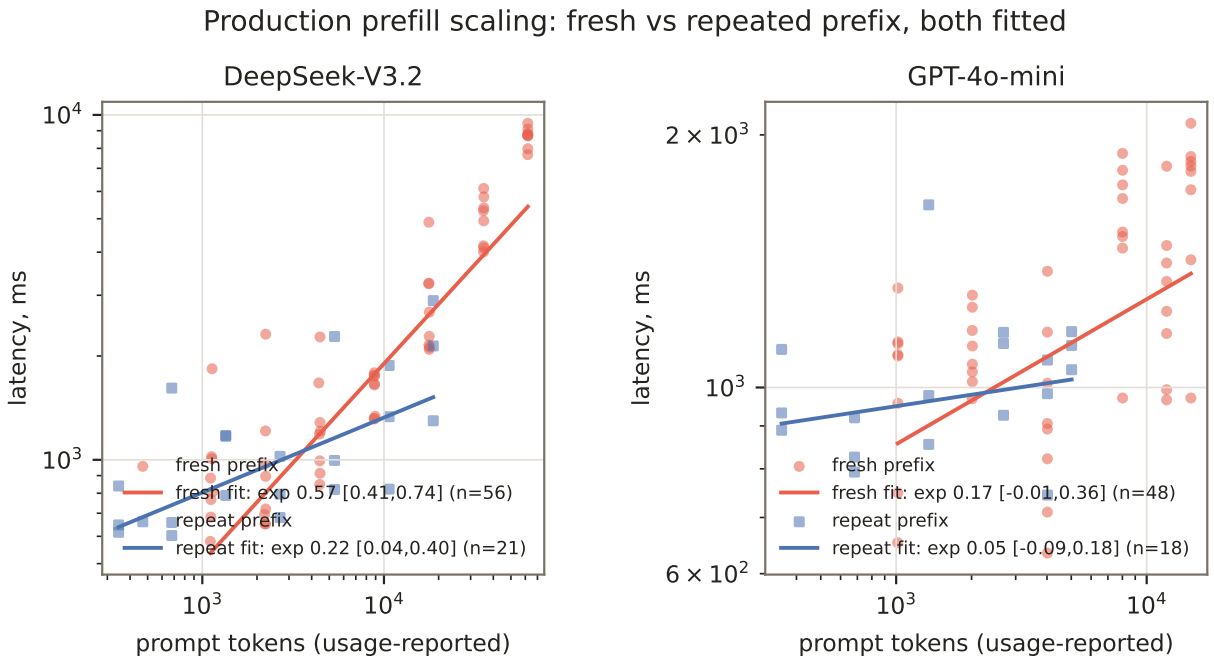


Figure 13. Production prefill scaling, powered follow-up: fresh-nonce (red) against repeated-prefix (blue), both fitted with the same log-log model. DeepSeek's fresh exponent is sub-quadratic (0.57, CI [0.41, 0.74], n=56); GPT-4o-mini's prefill is overhead-dominated at these spans (0.17, CI [-0.01, 0.36], n=48). Exponents constrain the attention node; they do not uniquely select a kernel (Fig. 9).

follow-up runs $n = 10$ fresh secrets $\times$ {baseline, secret-in-state, secret-in-sibling} in randomised order. Planting the secret in the shared state moves the answer by ++0.98; planting it in a sibling question moves it by +0.00; the permutation test gives $p = 2 \times 10^{-4}$ (Fig. 15). Isolated branches over a shared state are now an L2-grade measured property of this endpoint, not a three-trial hint.

9. API observability: how the surface bounds the boundary

The boundary is not fixed by nature. It is set by what the provider chooses to expose. This section measures that directly: how the identified set changes as channels are closed one by one.

9.1. Channel-closure ablation

We take one synthetic stack and keep the model fixed while closing surfaces in sequence: full access $\rightarrow$ a top-5 cap $\rightarrow$ logits closed $\rightarrow$ billing also closed $\rightarrow$ added timing noise. Table 12 shows how the detectors respond. Closing the logits weakens the wire-grid and repeat evidence (the dtype score drops from 0.76 to 0.40). Closing billing removes tokenizer acquisition entirely, because the count channel is gone. Closing both leaves the behavioural core intact (generation family, contract) but stops every numeric reconstruction. Adding timing noise at the tested level changed nothing further. This is a detector-response ablation on a single stack. It maps the *direction* of each dependency. It is not a universal proof that closing a channel hides a property from every possible estimator.

9.2. Access regimes, restated as provider choices

The three regimes of Sec. 2.3 are decisions a provider makes: how deep the logprobs go, whether `logit_bias` is allowed, how finely streaming is chunked, how detailed the usage report is, whether errors are standardised, and how smooth the timing is. Each lever moves the boundary (Table 12). A provider that limits top-k to top-5 moves width from identified to access-limited. One that buffers its streaming removes the decode-state channel. One that standardises its error messages removes contract reads. The same levers also shape legitimate audit surfaces, so this cuts both

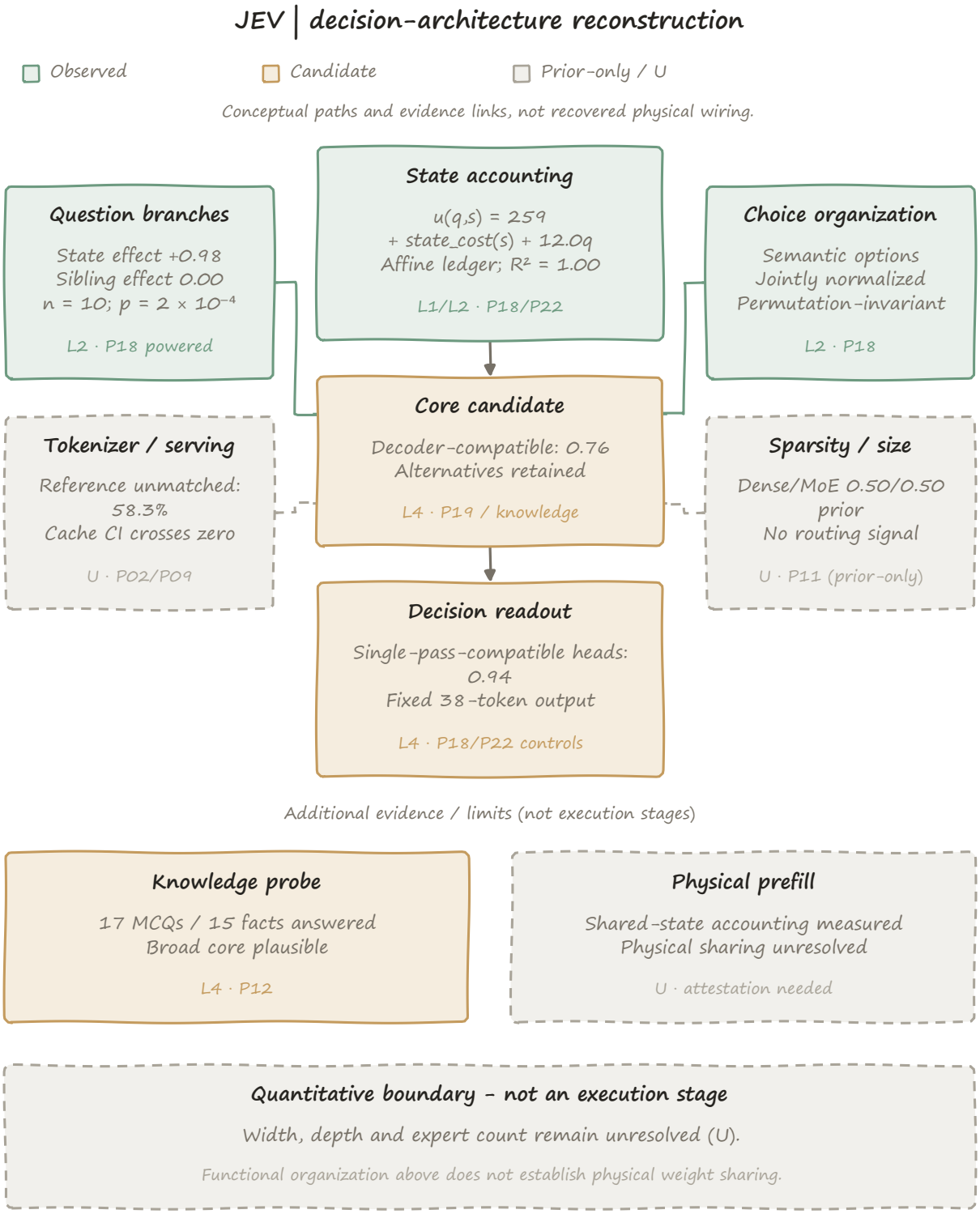


Figure 14. JEV's decision reconstruction: measured state accounting, isolated branches, and normalized choices alongside a candidate core and single-pass-compatible readout. Side links associate evidence; neither these links nor the affine ledger establish physical weight sharing. The quantitative boundary remains unresolved.

ways: *API surface design is observability policy.* Transparency obligations are a reason to document what each choice hides and what it legitimately protects [4, 5]. Our boundary ledger (Table 13) states the current position for each property.

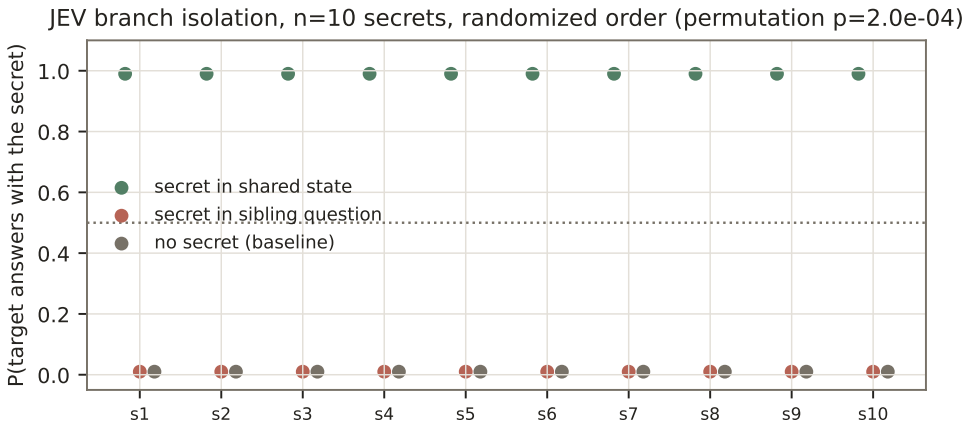


Figure 15. JEV branch isolation, powered: n=10 fresh secrets, randomised order. Secrets in the shared state are used (+0.98); secrets in sibling questions are invisible (0/10, permutation $p = 2 \times 10^{-4}$).

Table 12. Same-model channel closure: settled-property count and key detector scores as surfaces close. "–" = channel absent, property unmeasurable.

Surface	Scores $\geq .65$	Wire dtype score	Cache score	Tokenizer estimate
full	17	0.76	0.86	available
cap5	17	0.73	0.86	available
-logit	15	0.40	0.86	available
-bill	13	0.40	0.50	not measured
-time	13	0.40	0.50	not measured

9.3. What the endpoint does not tell you

The layer model plus the closure data justify the paper's negative-inference discipline. We repeat it here because it is doing real work in every result: *recall is not attention; latency is not kernel complexity; behaviour is not a training recipe; a prior-only candidate is not recovered structure; an unchanged API is not evidence of an unchanged model.* The counterpoint to that last worry is the swap fixture in Sec. 7.3, whose detection rates bound it empirically for large designed changes.

10. The identification boundary, consolidated

The stopping map (Table 13) is the paper's second product. Each open component is labelled **I** (identified), **C** (cost-limited: a nominal request scale computed from $\hat{\kappa}$), **A** (access-limited: the channel is not exposed on this interface), or **N** (non-identifiable: a scoped witness). This replaces both overclaiming and vague caution with a boundary you can act on. Sec. 9 shows that the boundary moves when the provider changes its surface. A partial identification is a positive result. The identified components stay recovered. The open ones carry their class and either a nominal cost or a named next experiment. An instrument that reported everything as certain would tell you less, not more.

The boundary is mostly not where intuition says. The naive expectation is "tokenizers are visible, everything deeper is guesswork." It is wrong in both directions. Output geometry can be recovered to the exact width with a modest query budget whenever the access regime allows it. Meanwhile two production tokenizer variants are *provably* inseparable over ordinary text. The map, not the motto, is the contribution.

Table 13. The boundary ledger, consolidated. **I** identified; **C** cost-limited (measured $\hat{\kappa}$; nominal cost scale); **A** access-limited (the channel is not exposed on this interface); **N** non-identifiable (scoped witness).

Target	Class	Why / next
o200k variant (base vs harmony), plain text	N	Prop. 1; enable the special-token domain
Width beyond logit rank	N	Prop. 2, demonstrated 768 vs 1024 with $\max \lvert\Delta\ell\rvert = 0$; needs a full-rank unembedding assumption
Depth under identity insertion, behaviour-only	N	toy witness (App. A); a timing or channel change is required
Production width via top-5 APIs	A	rank capped by the column set; needs rich access or a validated restricted estimator
Streaming-decode state on buffered endpoints	A	DeepSeek's gateway buffers SSE; GPT-4o-mini exposes engine TBT
Transformer vs SSM on high-overhead endpoints	C	$\hat{\kappa}$ small at $\leq$12k context spans (Table 4); longer spans or co-located timing
Production depth	C	a validated compute/queueing model with independent hardware controls; timing $\hat{\kappa}$ characterized
Expert count / routing	A	a richer routing channel or vendor attestation
Positional scheme	C	scheme-specific extrapolation probes
Compute dtype (vs wire dtype)	C	a transport-aware model; the wire grid is already recovered at 80% (25 subjects)
Exact vocabulary	A	ID-space sweep; reference gap quantified (DeepSeek: 465 entries)
JEV physical readout implementation	A	controlled implementations or attestation; the functional readout is already L2

11. Related work

Classical system identification separates four jobs: designing experiments, selecting a model structure, estimating its parameters, and validating the result [6]. Information-based active learning and Bayesian experimental design give a principled reason to prefer discriminating requests [7, 8]. Adaptive guarantees need assumptions we do not claim [9].

Model extraction and geometry. Extraction attacks clone a model's function [10, 11, 12, 13]. Cryptanalytic and numeric methods recover weights or output structure when a specific kind of access is available [14, 15, 16, 17, 18]. NightVision estimates width, depth, and parameter count under restrictive APIs on 32 models [19]; [20] show the limits of token-count lineage; token inflation motivates auditing the ledger [1]. The conceptual difference matters. NightVision asks *can restrictive API measurements recover hidden dimensions, depth, and parameter counts.* It is an estimator. This paper asks *what functional properties are identifiable under a given API observation channel*, and its output is the identified set $[\omega]_{\mathcal{C},\mathcal{X}}$, the per-property identified sets, and a measured boundary, not a point estimate. Concurrent 2026 work develops identified-set geometry for top-k-censored logprob access, which is the mathematical cousin of our Prop. 2. Our geometry module is a budgeted-coverage variant of that family. Sec. 7.1 runs a NightVision-style estimator on our subjects for a same-data comparison, and Prop. 2 pins down the formal limit that both approaches share.

Fingerprinting and behavioural probes. LLMmap identifies known models from fixed query sets [21]. Behavioural fingerprints compare endpoints [22]. Concurrent 2026 work derives single-token behavioural distributions for fingerprinting and verification, and agentic-surface identification (LIDAR-style) extends the same observability question beyond text. These methods answer "which known model is this" (or "which agent stack"), not "what architecture does an unknown model

have." The two tasks compose naturally: fingerprint first, then reconstruct whatever is unfamiliar. Long-context and behavioural evaluation [23, 24, 25, 26, 27] motivate our controlled measurements.

Candidate mechanisms. The hypothesis space is spanned by transformer variants [28, 29, 30, 31, 32], MoE [33, 34, 35], MLA [36], sparse attention [3], SSMs [37, 38, 39], and serving systems [40, 41, 42, 43].

Position. The closest works are NightVision (dimension estimation under a restricted API), LLMmap (identification of known models), and the concurrent identified-set and fingerprinting analyses above. None of these neighbours closes the gap, because the contribution here is the *combination*: active probing across heterogeneous API channels (usage, logprob geometry, timing, behaviour); functional rather than weight-level architecture; identified sets with proved non-identifiability witnesses; a blind commit-reveal detector benchmark that prices the reliability of every claim; and a channel-closure ablation showing that the boundary moves with the provider's surface (Sec. 9). What that combination buys is a unified, evidence-graded partial reconstruction with a measured reliability per claim and a proved boundary. That is the gap this paper fills.

12. Limitations

- **Serving realism.** ForensicsBench runs on CPU containers. GPU kernels, real vLLM batching, and quantization-aware serving all change the timing channels. The decode-state discriminator is validated on our hardware. Production spans ($\leq$12–56k tokens) are drowned in noise, so production family splits stay U.
- **MoE and depth are the weak channels.** Domain timing reaches only 88% on the benchmark. Depth inversion needs a calibrated compute and queueing model that we do not have for production hardware.
- **Score semantics.** Support scores are heuristic rankings. Their calibration is measured (macro-averaged ECE 0.249 [0.153, 0.351]), but they are not probabilities, and correlated evidence groups are averaged rather than modelled jointly. The largest prior-dependence (attention/state: 90% with the reference prior against 38% evidence-only) is reported as a result, not hidden (Sec. 6).
- **Scale.** The widest resolved geometry is 2,048 (as a bound) and 896 (as a resolved point). Frontier widths need a bigger budget. The blind fixtures cover large, designed changes; subtle drift and adversarial spoofing are untested.
- **Release.** The repository accompanies this paper (title page). The DOI is pending, and provider notification and terms-of-service review remain the authors' external obligations (Sec. 13).

13. Ethics, dual use, and release status

All production measurements used configured user accounts with bounded budgets. No private tenants, credentials, training data, or weights were accessed. Architecture forensics supports auditing and procurement verification, but the same techniques can help with model theft or proprietary profiling. The artifact therefore ships with rate limits and authentication requirements for any multi-tenant deployment. Having public API access is not the same as having permission to publish. Provider notification, terms review, and, where appropriate, operator consent are documented by the author before public release, and named-service results can be anonymized on request. The ForensicsBench journals, harness, and frozen protocol are released with this paper. The JEV prior public study is archived with a content hash.

14. Conclusion

A production API is a measurement surface, and its resolution is something you can measure and prove, not something you have to guess. On ground truth, controlled requests reconstruct the contract, the tokenizer count-family, the autoregressive-compatible generation contract, the

transformer/SSM split, the output bottleneck (within budget, with a coverage curve), and the wire numerics. They expose MoE only weakly and depth not at all. Every remaining hole is one of three things: witnessed non-identifiable, cost-characterized through $\hat{\kappa}$, or one named experiment away. Architecture reconstruction from API access is neither impossible nor unbounded. It is a bounded capability with a map. ForensicsBench, the blind fixtures, the evaluated evidence-ranking scores, and the witnesses are our attempt to draw that map clearly enough that others can extend it.

Appendix A. Reproduction, protocol, and manifests

The frozen battery protocol is pinned by its hash (`31fc86fdc7fa99a8`). Every subject journal (raw requests, responses, and timings) is under `runs/forensics-bench/` and in the volume manifest. Detector failures found during development, and their fixes, are logged in `paper/bench/out/`. There were four: a curvature gate (P19) that fired on numerically-zero curvature because it had no effect-size check; a units error in the decode-growth slope conversion (P23); a per-round coupling that compounded on itself; and a grid-statistic error in the first blind-fixture detector. The paper reports the repaired instrument, and the failure log is part of the scientific record.

The toy depth witness works like this. If the configuration space Ω allows both f and $f \circ \mathrm{Id}$ (the same function with an identity layer inserted), with identical wrappers and identical output laws, then depth cannot be identified through behaviour alone. Inserting an identity layer leaves $f(x)$ unchanged while increasing the declared depth.

Table A.14. ForensicsBench subject roster with ground truth as recorded from loaded configs.

Key	Family	Width	Layers	Params	Axis / notes
`bert-mini`	encoder_only	256	4	11.2M	core
`bert-small`	encoder_only	512	4	28.8M	core
`bert-tiny`	encoder_only	128	2	4.42M	core
`distilbert`	encoder_only	768	6	66.99M	core
`distilgpt2`	decoder_only	768	6	81.91M	core
`gpt2`	decoder_only	768	12	124.44M	core
`gpt2-batch8`	decoder_only	768	12	124.44M	batched
`gpt2-bf16`	decoder_only	768	12	124.44M	bf16
`gpt2-int8`	decoder_only	768	12	124.44M	int8_dynamic
`gpt2-wirebf16`	decoder_only	768	12	124.44M	wire-bf16
`mamba-130m`	ssm_or_linear	768	24	129.14M	core
`mamba-370m`	ssm_or_linear	1024	48	371.52M	core
`olmoe-1b7b`	decoder_only	2048	16	6919.16M	core
`opt-125m`	decoder_only	768	12	125.24M	core
`pythia-14m`	decoder_only	128	6	14.07M	core
`pythia-160m`	decoder_only	768	12	162.32M	core
`pythia-31m`	decoder_only	256	6	30.49M	core
`pythia-70m`	decoder_only	512	6	70.43M	core
`qwen25-05b`	decoder_only	896	24	494.03M	core
`router-gpt2-pythia`	router_or_cascade	768	12	124.44M	router (two checkpoints, one name)
`smollm-135m`	decoder_only	576	30	134.52M	core
`t5-small`	encoder_decoder	512	6	60.51M	core
`tiny-dsv3-mla`	decoder_only	256	4	32.15M	mechanics (random init)
`tiny-mixtral-moe`	decoder_only	256	4	38.98M	mechanics (random init)
`tinyllama-1b`	decoder_only	2048	22	1100.05M	core

Appendix B. Calibration detail

Reliability bins (mean confidence against empirical accuracy) per property for the group-aware ranker are in `bench/out/bench_metrics.json`; the legacy independent-factor ranker is the

sensitivity column. The evidence-only view re-runs the identical updates starting from uniform priors.

Appendix C. Blind fixture hash chain

`bench/out/blind_fixtures.commits.json` holds the SHA-256 commitments written before each scan; `blind_fixtures.json` holds the labels, salts, detections, and per-episode `commit_ok` flags. Recomputing `sha256(salt|label)` verifies the chain.

Appendix D. Production run provenance

Production scans: `runs/20260926-221747-gpt-4o-mini`, `runs/final/20260926-210641-deepseek`, `runs/20260926-221526-jev`; follow-ups: `runs/production-followup/`, `runs/revision-controls/`, `runs/jev-v4/`. Costs are request and token counts per run; dollar equivalents are not verified bills. All 43 cited arXiv IDs were checked against the arXiv API (`bibliography_audit.json`).

References

[1] L. Chen, L. Zhang, C. Tang, P. Sun, J. Lai, Y. Huang, Z. Zhang, X. Shen, The more it says, the more you pay: A black-box audit of provider-side token inflation in LLM services (2026). arXiv:2609.20370.

[2] OpenAI, GPT-4o mini: Model documentation, https://platform.openai.com/docs/models/gpt-4o-mini, living documentation; retrieved 29 September 2026. Listed year refers to model release (2024).

[3] DeepSeek-AI, Deepseek-v3.2: Pushing the frontier of open large language models, arXiv preprint, arXiv:2512.02556 (2025).

[4] European Parliament and Council, Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act), Official Journal of the European Union (2024).

[5] R. Bommasani, K. Klyman, S. Longpre, S. Kapoor, N. Maslej, B. Xiong, D. Zhang, P. Liang, The foundation model transparency index, arXiv preprint, arXiv:2310.12941 (2023).

[6] L. Ljung, System Identification: Theory for the User, 2nd Edition, Prentice Hall, 1999.

[7] D. J. C. MacKay, Information-based objective functions for active data selection, Neural Computation 4 (4) (1992) 590–604.

[8] K. Chaloner, I. Verdinelli, Bayesian experimental design: A review, Statistical Science 10 (3) (1995) 273–304.

[9] D. Golovin, A. Krause, Adaptive submodularity: Theory and applications in active learning and stochastic optimization, JAIR, arXiv:1003.3967 (2011).

[10] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, T. Ristenpart, Stealing machine learning models via prediction apis, USENIX Security Symposium, arXiv:1609.02943 (2016).

[11] T. Orekondy, B. Schiele, M. Fritz, Knockoff nets: Stealing functionality of black-box models, CVPR, arXiv:1812.02766 (2019).

[12] K. Krishna, G. S. Tomar, A. P. Parikh, N. Papernot, Thieves on sesame street! model extraction of bert-based apis, ICLR, arXiv:1910.12366 (2020).

[13] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, A. Swami, Practical black-box attacks against machine learning, ACM Asia CCS, arXiv:1602.02697 (2017).

[14] N. Carlini, M. Jagielski, I. Mironov, Cryptanalytic extraction of neural network models, CRYPTO, arXiv:2003.04884 (2020).

[15] D. Rolnick, K. P. Kording, Reverse-engineering deep relu networks, ICLR, arXiv:1910.00744 (2020).

[16] N. Carlini, D. Paleka, K. D. Dvijotham, T. Steinke, J. Hayase, A. F. Cooper, K. Lee, M. Jagielski, M. Nasr, A. Conmy, I. Yona, E. Wallace, D. Rolnick, F. Tramèr, Stealing part of a production language model, ICML, arXiv:2403.06634 (2024).

[17] M. Finlayson, X. Ren, S. Swayamdipta, Logits of api-protected llms leak proprietary information, ICML, arXiv:2403.09539 (2024).

[18] A. Naseh, K. Krishna, M. Iyyer, A. Houmansadr, Stealing the decoding algorithms of language models, ACM CCS, arXiv:2303.04729 (2024).

[19] C. Ellis, S. Chaudhari, M.-Y. Wang, L. Barnes, G. Fanti, J. M. F. Moura, Black-box inference of llm architectural properties with restrictive api access, arXiv preprint, arXiv:2607.01313 (2026).

[20] B. Chen, Token counts are not model lineage: A frozen-threshold holdout study of black-box LLM API fingerprinting (2026). arXiv:2608.29930.

[21] D. Pasquini, E. M. Kornaropoulos, G. Ateniese, Llmmap: Fingerprinting for large language models, USENIX Security Symposium, arXiv:2407.15847 (2025).

[22] Z. Pei, H.-L. Zhen, Y. Zhang, Z. Yang, X. Li, X. Yu, M. Yuan, B. Yu, Behavioral fingerprinting of large language models (2025). arXiv:2509.04504.

[23] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, B. Ginsburg, Ruler: What's the real context size of your long-context language models?, COLM, arXiv:2404.06654 (2024).

[24] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, Lost in the middle: How language models use long contexts, TACL, arXiv:2307.03172 (2024).

[25] S. Casper, C. Ezell, C. Siegmann, N. Kolt, T. L. Curtis, B. Bucknall, A. Haupt, K. Wei, J. Scheurer, M. Hobbhahn, L. Sharkey, S. Krishna, M. V. Hagen, S. Alberti, A. Chan, Q. Sun, M. Gerovitch, D. Bau, M. Tegmark, D. Krueger, D. Hadfield-Menell, Black-box access is insufficient for rigorous ai audits, ACM FAccT, arXiv:2401.14446 (2024).

[26] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, X. Xie, A survey on evaluation of large language models, ACM Transactions on Intelligent Systems and Technology, arXiv:2307.03109 (2024).

[27] T. Shevlane, S. Farquhar, B. Garfinkel, M. Phuong, J. Whittlestone, J. Leung, D. Kokotajlo, N. Marchal, M. Anderljung, N. Kolt, L. Ho, D. Siddarth, S. Avin, W. Hawkins, B. Kim, I. Gabriel, V. Bolina, J. Clark, Y. Bengio, P. Christiano, A. Dafoe, Model evaluation for extreme risks, arXiv preprint, arXiv:2305.15324 (2023).

[28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, NeurIPS, arXiv:1706.03762 (2017).

[29] N. Shazeer, Fast transformer decoding: One write-head is all you need, arXiv preprint, arXiv:1911.02150 (2019).

[30] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, Gqa: Training generalized multi-query transformer models from multi-head checkpoints, EMNLP, arXiv:2305.13245 (2023).

[31] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, Y. Liu, Roformer: Enhanced transformer with rotary position embedding, Neurocomputing, arXiv:2104.09864 (2024).

[32] O. Press, N. A. Smith, M. Lewis, Train short, test long: Attention with linear biases enables input length extrapolation, ICLR, arXiv:2108.12409 (2022).

[33] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, J. Dean, Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, ICLR, arXiv:1701.06538 (2017).

[34] W. Fedus, B. Zoph, N. Shazeer, Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, JMLR, arXiv:2101.03961 (2022).

[35] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, Mixtral of experts, arXiv preprint, arXiv:2401.04088 (2024).

[36] DeepSeek-AI, Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, arXiv preprint arXiv:2405.04434 (2024).

[37] A. Gu, K. Goel, C. Ré, Efficiently modeling long sequences with structured state spaces, arXiv preprint arXiv:2111.00396 (2022).

[38] A. Gu, T. Dao, Mamba: Linear-time sequence modeling with selective state spaces, arXiv preprint arXiv:2312.00752 (2024).

[39] T. Dao, A. Gu, Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality, arXiv preprint arXiv:2405.21060 (2024).

[40] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré, Flashattention: Fast and memory-efficient exact attention with io-awareness, NeurIPS, arXiv:2205.14135 (2022).

[41] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with pagedattention, SOSP, arXiv:2309.06180 (2023).

[42] Y. Leviathan, M. Kalman, Y. Matias, Fast inference from transformers via speculative decoding, ICML, arXiv:2211.17192 (2023).

[43] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, J. Jumper, Accelerating large language model decoding with speculative sampling, arXiv preprint, arXiv:2302.01318 (2023).